

Plataforma Cyberlab
Curso: 2º DAW / Online

Javier Perea Gómez
Tutor: Ignacio Poveda Salinas

Dedicatoria

A mi familia y mi pareja por apoyarme durante el curso y el desarrollo de este proyecto.

INDICE

1. ABSTRACT 6

1.1 Resumen (Español) 6

1.2 Resume (English) 7

2. JUSTIFICACIÓN DEL PROYECTO 8

2.1 Motivación 8

2.2 Estado de la cuestión: plataformas similares 8

2.3 Público objetivo 9

3. INTRODUCCIÓN 11

3.1 Descripción general de la plataforma 11

3.2 Principales funciones 11

3.3 Requisitos del sistema 12

4. OBJETIVOS 13

4.1 Listado de objetivos 13

5. RFTP: Requisitos, Funciones, Tareas y Pruebas 13

5.1 R01 — Laboratorios interactivos 14

5.2 R02 — Diccionario de términos 17

5.3 R03 — Medidas de seguridad 18

6. DESCRIPCIÓN 19

6.1 Arquitectura de la solución 19

7. Casos de uso 22

7.1 Caso de uso CU-01: Explotar vulnerabilidad SQL Injection en modo vulnerable 22

7.2 Caso de uso CU-02: Explotar vulnerabilidad XSS en modo vulnerable 24

7.3 Caso de uso CU-03: Explotar vulnerabilidad JWT (algoritmo none) en modo vulnerable . 26

7.4 Caso de uso CU-04: Explotar vulnerabilidad IDOR en modo vulnerable 28

7.5 Caso de uso CU-05: Buscar términos en el diccionario de ciberseguridad 30

8. DISEÑOS 32

8.1 Diagrama de la base de datos *32*

8.2 Diagrama E/R (Entidad-Relación) *33*

8.3 Diagrama de flujo de navegación *34*

8.4 Interfaces de usuario *35*

9. Descripción detallada de los laboratorios 39

9.1 Laboratorio SQL Injection (SQLi) *39*

9.2 Laboratorio Cross-Site Scripting (XSS) *44*

9.3 Laboratorio Broken Authentication *46*

9.4 Laboratorio IDOR (Insecure Direct Object Reference) *48*

9.5 Laboratorio SSRF (Server-Side Request Forgery) *50*

..... 51

9.6 Laboratorio JWT Attacks *52*

9.7 Diagrama de red *56*

10. TECNOLOGÍAS UTILIZADAS 57

11. METODOLOGÍA 59

11.1 Metodología de desarrollo *59*

11.2 Planificación inicial estimada (antes de la implementación) *60*

11.3 Planificación real ejecutada (tiempo dedicado finalmente) *61*

11.4 Análisis de desviaciones *62*

11.5 Diagrama de Gantt *63*

11.6 Presupuesto estimado *64*

11.7 Despliegue e instalación *66*

12. TRABAJOS FUTUROS 72

12.1. Nuevos laboratorios planificados 72

12.2. Sistema de progreso y gamificación 73

12.3. Mejoras técnicas y de despliegue 73

13. CONCLUSIONES 74

14. REFERENCIAS 76

1. ABSTRACT

1.1 Resumen (Español)

El presente Trabajo de Fin de Grado describe el diseño y desarrollo de CYBERLAB, una plataforma web educativa e interactiva orientada al aprendizaje de las principales vulnerabilidades de seguridad en aplicaciones web. La plataforma implementa laboratorios prácticos basados en las vulnerabilidades más relevantes del estándar OWASP Top 10, permitiendo al usuario experimentar en tiempo real la diferencia entre código vulnerable y código seguro mediante un sencillo conmutador de modo integrado en cada laboratorio.

CYBERLAB ha sido desarrollado sobre Node.js con el framework Express, utilizando el motor de plantillas EJS para la generación de vistas dinámicas y Sequelize como ORM para la gestión de la base de datos relacional. La plataforma incluye laboratorios dedicados a SQL Injection, Cross-Site Scripting (XSS), Broken Authentication, Insecure Direct Object Reference (IDOR), Server-Side Request Forgery (SSRF) y ataques JWT, así como un diccionario interactivo de términos de ciberseguridad con más de cuarenta entradas, filtrado por categorías y búsqueda en tiempo real implementado con React.

El objetivo principal del proyecto es proporcionar un entorno controlado y seguro donde estudiantes y profesionales en formación puedan comprender el funcionamiento real de estas vulnerabilidades, sus vectores de explotación y las medidas de mitigación asociadas, contribuyendo así a la concienciación y a la formación práctica en materia de seguridad informática. Todo el código fuente relevante de cada laboratorio es visible y explicado dentro de la propia plataforma, eliminando la necesidad de consultar documentación externa durante el aprendizaje.

1.2 Resume (English)

This Final Degree Project describes the design and development of CYBERLAB, an interactive educational web platform focused on teaching the main web application security vulnerabilities. The platform implements hands-on labs based on the most relevant OWASP Top 10 vulnerabilities, allowing users to experience in real time the difference between vulnerable and secure code through a simple mode toggle built into each lab.

CYBERLAB has been built on Node.js with the Express framework, using the EJS template engine for dynamic view rendering and Sequelize as the ORM for relational database management. The platform includes dedicated labs for SQL Injection, Cross-Site Scripting (XSS), Broken Authentication, Insecure Direct Object Reference (IDOR), Server-Side Request Forgery (SSRF) and JWT attacks, as well as an interactive cybersecurity dictionary with over forty entries, category filtering and real-time search implemented with React.

The main goal of the project is to provide a controlled and safe environment where students and professionals in training can understand the real-world mechanics of these vulnerabilities, their exploitation vectors and the associated mitigation measures, thereby contributing to cybersecurity awareness and practical training. All relevant source code for each lab is visible and explained within the platform itself, eliminating the need to consult external documentation during the learning process.

2. JUSTIFICACIÓN DEL PROYECTO

2.1 Motivación

La ciberseguridad se ha convertido en uno de los pilares fundamentales del desarrollo de software moderno. Sin embargo, la formación práctica en este ámbito continúa siendo escasa en los currículos de los ciclos formativos de grado superior, donde el énfasis recae habitualmente en el desarrollo de aplicaciones funcionales sin profundizar en los riesgos de seguridad asociados a cada decisión de implementación.

La motivación principal que impulsa la creación de CYBERLAB reside en la necesidad de disponer de una herramienta educativa accesible, íntegramente en castellano y con un enfoque completamente práctico, que permita a cualquier estudiante o profesional en formación entender de manera visual e interactiva cómo funcionan las vulnerabilidades web más comunes. La mayoría de plataformas similares existentes están en inglés, requieren instalaciones complejas o están orientadas a perfiles ya avanzados en ciberseguridad, lo que supone una barrera de entrada significativa para el público al que se dirige este proyecto.

Adicionalmente, el auge de los ciclos de ciberataques a empresas españolas en los últimos años —con incidentes de alto impacto como los sufridos por el SEPE, el Hospital Clínic de Barcelona o el Ayuntamiento de Sevilla— pone de manifiesto la urgencia de formar a los futuros desarrolladores con una mentalidad de seguridad desde el diseño (security by design), no como un añadido posterior.

2.2 Estado de la cuestión: plataformas similares

Existen diversas plataformas educativas orientadas a la ciberseguridad práctica. A continuación, se presenta una comparativa razonada con las más relevantes, justificando el hueco que cubre CYBERLAB:

Plataforma	Características	Diferencia con CYBERLAB
DVWA (Damn Vulnerable Web Application)	Aplicación PHP clásica con labs de SQLi, XSS, CSRF. Requiere instalación LAMP/XAMPP. Sin interfaz moderna ni explicaciones integradas.	CYBERLAB funciona sobre Node.js, sin dependencias de servidor web adicionales. Incluye explicaciones y código visible en la propia interfaz.
WebGoat (OWASP)	Plataforma Java de OWASP con lecciones interactivas. Completa pero compleja de instalar. Documentación solo en inglés.	CYBERLAB está completamente en castellano y tiene una curva de aprendizaje mínima.
HackTheBox / TryHackMe	Plataformas gamificadas para CTFs y pentesting. Excelentes para práctica avanzada pero con curva muy elevada. De pago para acceso completo.	CYBERLAB es gratuito, de código abierto y diseñado para la iniciación, no para perfiles avanzados.
PortSwigger Web Security Academy	Referencia del sector. Labs web muy bien documentados. Solo en inglés. No muestra el código fuente vulnerable directamente.	CYBERLAB muestra el código fuente real del servidor en cada laboratorio, con explicación técnica en castellano.
CYBERLAB (este proyecto)	Node.js/Express, castellano, modo vulnerable/seguro en tiempo real, código fuente visible, diccionario integrado, sin instalación compleja.	Es la propia propuesta diferencial del proyecto.

2.3 Público objetivo

CYBERLAB está dirigido principalmente a los siguientes perfiles:

- Alumnos de ciclos formativos de grado superior en Desarrollo de Aplicaciones Web (DAW), Desarrollo de aplicaciones multiplataforma (DAM) y Administración de Sistemas Informáticos en Red (ASIR), especialmente en los módulos de desarrollo web, seguridad informática y servicios de red.

- Desarrolladores web junior que deseen iniciarse en la seguridad ofensiva y defensiva sin necesidad de conocimientos previos especializados en ciberseguridad.
- Docentes de ciclos formativos que busquen una herramienta de demostración práctica para ilustrar vulnerabilidades web en el aula.
- Cualquier persona con conocimientos básicos de programación web que quiera entender cómo funcionan los ataques más comunes y cómo prevenirlos.

3. INTRODUCCIÓN

3.1 Descripción general de la plataforma

CYBERLAB es una aplicación web educativa que permite al usuario aprender sobre vulnerabilidades de seguridad web de forma práctica e interactiva. La plataforma ofrece laboratorios individuales para cada tipo de vulnerabilidad, donde el usuario puede alternar entre un modo vulnerable —que reproduce el comportamiento inseguro real tal y como aparecería en una aplicación mal implementada— y un modo seguro —que muestra cómo debería implementarse correctamente la misma funcionalidad siguiendo las buenas prácticas del sector—.

Cada laboratorio expone el código fuente relevante con su explicación técnica, permitiendo al usuario comprender no solo el vector de ataque sino también la solución de mitigación aplicable. Adicionalmente, la plataforma incorpora un diccionario interactivo de términos de ciberseguridad con más de cuarenta entradas organizadas por categorías (ataques web, red, reconocimiento, ingeniería social, malware y conceptos generales) y con capacidad de búsqueda en tiempo real.

La aplicación está construida sobre una arquitectura MVC (Modelo-Vista-Controlador) con Node.js y Express en el backend, EJS como motor de plantillas para las vistas, y Sequelize como ORM para la gestión de la base de datos. Los laboratorios están contenidos en módulos independientes, lo que facilita la incorporación de nuevos laboratorios sin necesidad de modificar el código existente.

3.2 Principales funciones

- Laboratorios interactivos de SQL Injection, XSS, Broken Authentication, IDOR, SSRF y JWT Attacks, cada uno con su propia vista, lógica de negocio y explicación técnica.
- Modo vulnerable / seguro conmutable en tiempo real mediante un botón en cada laboratorio, sin necesidad de reiniciar el servidor ni recargar la aplicación.

- Visualización del código fuente vulnerable y seguro con explicación técnica detallada en una vista dedicada accesible desde cada laboratorio.
- Alertas visuales diferenciadas (banners de color rojo) cuando la vulnerabilidad ha sido explotada con éxito, con indicación del vector de ataque utilizado.
- Diccionario de ciberseguridad con búsqueda en tiempo real y filtros por categoría, implementado como componente React embebido en la vista EJS sin necesidad de bundler.
- Navegación centralizada con layout compartido mediante express-ejs-layouts, que incluye barra de menú superior y pie de página en todas las vistas.
- Medidas de seguridad en la propia plataforma: cabeceras HTTP mediante Helmet, gestión de sesiones con express-session, control de peticiones con CORS y rate limiting de 500 req / 15 minutos.

3.3 Requisitos del sistema

- La plataforma debe ejecutarse en un entorno Node.js (versión 18 o superior) con acceso a una base de datos relacional compatible con Sequelize (MySQL, PostgreSQL o SQLite).
- Los laboratorios deben ser independientes entre sí y no requerir autenticación global para acceder a ellos.
- El modo vulnerable debe reproducir fielmente el comportamiento inseguro sin comprometer la integridad del servidor real ni del resto de laboratorios.
- La interfaz de usuario debe ser intuitiva y accesible para usuarios sin conocimientos previos de ciberseguridad, con guías de explotación incluidas en cada laboratorio.
- El código fuente de cada laboratorio debe ser visible y comprensible para el usuario desde la propia interfaz, diferenciando claramente el código vulnerable del seguro.

- La plataforma debe aplicar medidas de seguridad básicas en su propia infraestructura para servir como ejemplo de buenas prácticas a nivel de configuración del servidor.

4. OBJETIVOS

4.1 Listado de objetivos

- Desarrollar una plataforma web educativa funcional sobre Node.js/Express que implemente laboratorios interactivos de las vulnerabilidades OWASP más comunes.
- Implementar un sistema de modo dual (vulnerable/seguro) en cada laboratorio que permita comparar en tiempo real ambos comportamientos sin reinicio del servidor.
- Proporcionar visualización del código fuente vulnerable y seguro con explicación técnica en cada laboratorio, directamente accesible desde la interfaz.
- Incluir un diccionario interactivo de términos de ciberseguridad con búsqueda y filtrado por categorías implementado con React.
- Aplicar medidas de seguridad básicas en la propia plataforma (Helmet, CORS, rate limiting, sesiones) como demostración de buenas prácticas.
- Crear una interfaz de usuario moderna y coherente mediante un layout compartido, con temática dark hacker y navegación centralizada.

5. RFTP: Requisitos, Funciones, Tareas y Pruebas

El siguiente desglose RFTP recoge los requisitos del sistema, sus funciones asociadas, las tareas de implementación y las pruebas planificadas para verificar cada funcionalidad.

5.1 R01 — Laboratorios interactivos

Código	Descripción
R01F01	El sistema debe implementar el laboratorio de SQL Injection con formulario de login y visualización de la query ejecutada.
R01F01T01	Crear la tabla lab_sqli_users en base de datos mediante Sequelize con campos id, email y password.
R01F01T01P01	Insertar datos de prueba y verificar que la tabla se crea correctamente al arrancar el servidor.
R01F01T02	Implementar la query vulnerable con concatenación directa del input del usuario en la cadena SQL.
R01F01T02P01	Introducir el payload ' OR 1=1 -- en el campo email en modo VULNERABLE y verificar que el login es exitoso sin contraseña válida.
R01F01T03	Implementar la query segura con parámetros reemplazados de Sequelize (:email, :password).
R01F01T03P01	Introducir el mismo payload ' OR 1=1 -- en modo SEGURO y verificar que el login es rechazado.
R01F02	El sistema debe implementar el laboratorio de XSS con campo de texto libre y renderizado diferenciado por modo.
R01F02T01	Implementar renderizado directo del input con la etiqueta EJS <%- %> en modo vulnerable.
R01F02T01P01	Introducir <script>alert('XSS')</script> en modo VULNERABLE y verificar que el script se ejecuta en el navegador.
R01F02T02	Implementar escape de caracteres HTML en el controller en modo seguro y renderizado con <%= %>.
R01F02T02P01	Introducir el mismo payload en modo SEGURO y verificar que aparece como texto plano sin ejecutarse.

R01F03	El sistema debe implementar el laboratorio de Broken Authentication con base de datos simulada de usuarios.
R01F03T01	Implementar el login vulnerable que acepta cualquier usuario sin verificar la contraseña.
R01F03T01P01	Iniciar sesión con usuario 'admin' y contraseña incorrecta en modo VULNERABLE y verificar que se concede acceso.
R01F03T02	Implementar login seguro con bcrypt.compare() y bloqueo automático tras 3 intentos fallidos.
R01F03T02P01	Realizar 3 intentos fallidos en modo SEGURO y verificar que la cuenta queda bloqueada.
R01F04	El sistema debe implementar el laboratorio de IDOR con perfiles de usuario accesibles por ID en URL.
R01F04T01	Simular base de datos con 3 usuarios (admin id=1, user id=2, guest id=3) con datos sensibles.
R01F04T01P01	Verificar que los 3 perfiles son accesibles desde la página principal del laboratorio.
R01F04T02	Permitir acceso a cualquier perfil mediante el parámetro ?id=N sin validación en modo vulnerable.
R01F04T02P01	Acceder a /labs/idor/profile?id=3 desde sesión de usuario id=1 en modo VULNERABLE y verificar que se muestra el perfil ajeno con banner de explotación.
R01F04T03	Bloquear el acceso a perfiles cuyo id no coincida con la sesión activa en modo seguro.
R01F04T03P01	Intentar el mismo acceso en modo SEGURO y verificar que el servidor devuelve 'Acceso denegado'.
R01F05	El sistema debe implementar el laboratorio de SSRF con formulario de petición de URL arbitraria.
R01F05T01	Implementar petición directa a la URL proporcionada por el usuario mediante axios sin validación en modo vulnerable.
R01F05T01P01	Introducir http://localhost:3000/labs/ssrf/secret en modo VULNERABLE y verificar que el servidor devuelve el flag interno.

R01F05T02	Implementar validación mediante blacklist de términos sospechosos (localhost, 127.0.0.1, admin, /secret) en modo seguro.
R01F05T02P01	Introducir la misma URL en modo SEGURO y verificar que el servidor devuelve 'Acceso denegado'.
R01F05T03	Crear los endpoints internos /secret, /secret/secret, /secret/secret/secret y /admin con sus flags.
R01F05T03P01	Verificar que cada endpoint devuelve su flag correspondiente al ser accedido internamente.
R01F06	El sistema debe implementar el laboratorio de JWT Attacks con panel de administración protegido.
R01F06T01	Generar un token JWT firmado con HS256 y rol 'user' al iniciar sesión correctamente.
R01F06T01P01	Iniciar sesión con user@cyberlab.com / password123 y verificar que se genera y muestra el token JWT.
R01F06T02	Aceptar tokens con algoritmo 'none' (sin firma) en modo vulnerable para permitir la escalada de privilegios.
R01F06T02P01	Modificar el payload del token cambiando role a 'admin' y el algoritmo a 'none', eliminar la firma y acceder al panel de administración en modo VULNERABLE.
R01F06T03	Forzar la verificación del algoritmo HS256 y la firma del token en modo seguro.
R01F06T03P01	Intentar el mismo ataque en modo SEGURO y verificar que el servidor devuelve 'Token inválido'.

5.2 R02 — Diccionario de términos

Código	Descripción
R02F01	Implementar el diccionario como componente React embebido en la vista EJS, accesible desde /Diccionario.
R02F01T01	Definir el dataset con más de 40 términos, cada uno con campo term, tag, category y def.
R02F01T01P01	Acceder a /Diccionario y verificar que se muestran correctamente todas las entradas del dataset.
R02F01T02	Implementar búsqueda en tiempo real que filtre por término, etiqueta y definición simultáneamente.
R02F01T02P01	Buscar 'SQL' y verificar que aparecen todas las entradas cuyo término, etiqueta o definición contenga esa cadena.
R02F01T03	Implementar filtros por categoría (General, Ataque Web, Red, Reconocimiento, Ingeniería Social, Malware).
R02F01T03P01	Seleccionar la categoría 'Malware' y verificar que únicamente aparecen las entradas de esa categoría.
R02F01T04	Implementar los botones de filtro dinámicamente a partir de las categorías únicas del dataset.
R02F01T04P01	Verificar que todos los botones de categoría se generan automáticamente y funcionan correctamente.

5.3 R03 — Medidas de seguridad

Código	Descripción
R03F01	Configurar Helmet en app.js para añadir cabeceras de seguridad HTTP a todas las respuestas.
R03F01T01	Añadir el middleware Helmet con CSP desactivado intencionalmente para no interferir con los labs.
R03F01T01P01	Verificar la presencia de cabeceras como X-Content-Type-Options y X-Frame-Options en DevTools.
R03F02	Configurar express-rate-limit para limitar las peticiones a 500 por IP en ventanas de 15 minutos.
R03F02T01	Añadir el middleware de rate limiting de forma global en app.js antes de los routers.
R03F02T01P01	Verificar mediante script automatizado que la petición número 501 recibe una respuesta HTTP 429.
R03F03	Configurar express-session con secreto, resave false y saveUninitialized true para gestionar el modo de cada laboratorio.
R03F03T01	Añadir el middleware de sesiones y verificar que el modo (vulnerable/secure) se persiste correctamente entre peticiones.
R03F03T01P01	Cambiar el modo en un laboratorio, navegar a otra página y volver a verificar que el modo se ha mantenido.

6. DESCRIPCIÓN

6.1 Arquitectura de la solución

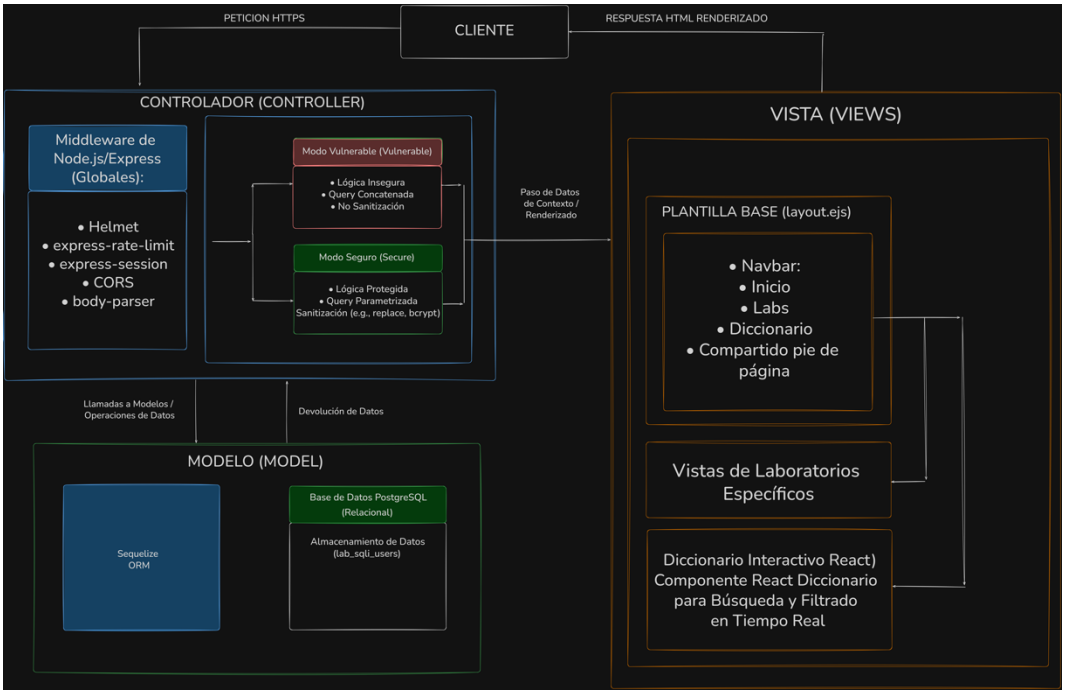
CYBERLAB sigue una arquitectura MVC (Modelo-Vista-Controlador) sobre Node.js. El servidor central (server.js) arranca la aplicación Express y establece la conexión con la base de datos mediante Sequelize antes de comenzar a escuchar peticiones. El fichero app.js actúa como punto de entrada de la aplicación, configurando los middlewares globales y registrando los routers de cada laboratorio y del diccionario.

Cada laboratorio está contenido en su propio directorio bajo /src/labs/, con un router.js que define las rutas HTTP específicas de ese laboratorio, un controller.js que implementa tanto la lógica vulnerable como la lógica segura, y las vistas EJS correspondientes bajo /src/views/labs/. Esta estructura modular garantiza la independencia entre laboratorios y facilita la incorporación de nuevos sin afectar al código existente.

El flujo de una petición típica en CYBERLAB es el siguiente: el navegador realiza una petición HTTP al servidor Express, que la enruta al router correspondiente según la URL, el router delega en el método apropiado del controller, el controller ejecuta la lógica de negocio (consultando la base de datos si es necesario) y renderiza la vista EJS con los datos calculados, que es devuelta al navegador como HTML.

Capa / Componente	Responsabilidad en CYBERLAB
server.js	Importa app.js, autentica la conexión con Sequelize, sincroniza el esquema de base de datos y arranca el servidor Express en el puerto configurado.
app.js	Configura Express: middlewares globales (Helmet, CORS, sesiones, rate limiting, body-parser), motor de vistas EJS, layouts compartidos, rutas de laboratorios y ruta 404.

labs/sqli/router.js + controller.js	Gestiona las rutas GET / (formulario), POST /login (procesamiento), GET /code (código) y GET /switch-mode (cambio de modo) del laboratorio SQLi.
labs/xss/router.js + controller.js	Gestiona las rutas del laboratorio XSS: formulario de input, procesamiento del submit, visualización de código y cambio de modo.
labs/broken-auth/router.js + controller.js	Gestiona el formulario de login de Broken Auth con lógica de verificación de contraseña (vulnerable vs. bcrypt) y bloqueo de cuenta.
labs/idor/router.js + controller.js	Gestiona la lista de perfiles, la vista de perfil individual con control de acceso (vulnerable vs. verificación de sesión) y el login simulado.
labs/ssrf/router.js + controller.js	Gestiona el formulario de petición de URL con la lógica de blacklist, los endpoints internos con flags y la vista de código.
labs/jwt/router.js + controller.js	Gestiona el login JWT, la generación de tokens HS256, el panel de administración con lógica de verificación de firma (vulnerable vs. estricta) y el código.
views/layout.ejs	Layout compartido con navegación superior (INICIO, LABORATORIOS, DICCIONARIO) y pie de página. Heredado por todas las vistas mediante express-ejs-layouts.
views/code.ejs	Vista genérica de código y explicación, compartida por todos los laboratorios. Recibe title, code, explanation y backLink del controller.
Diccionario/router.js + vista	Gestiona la ruta /Diccionario y renderiza el componente React del diccionario interactivo.
config/database.js	Configuración de la instancia de Sequelize con los parámetros de conexión a la base de datos desde variables de entorno.
public/css/style.css	Hoja de estilos global con temática dark hacker: variables CSS, tipografía Share Tech Mono / Rajdhani, grid de laboratorios, tarjetas, alertas y botones.

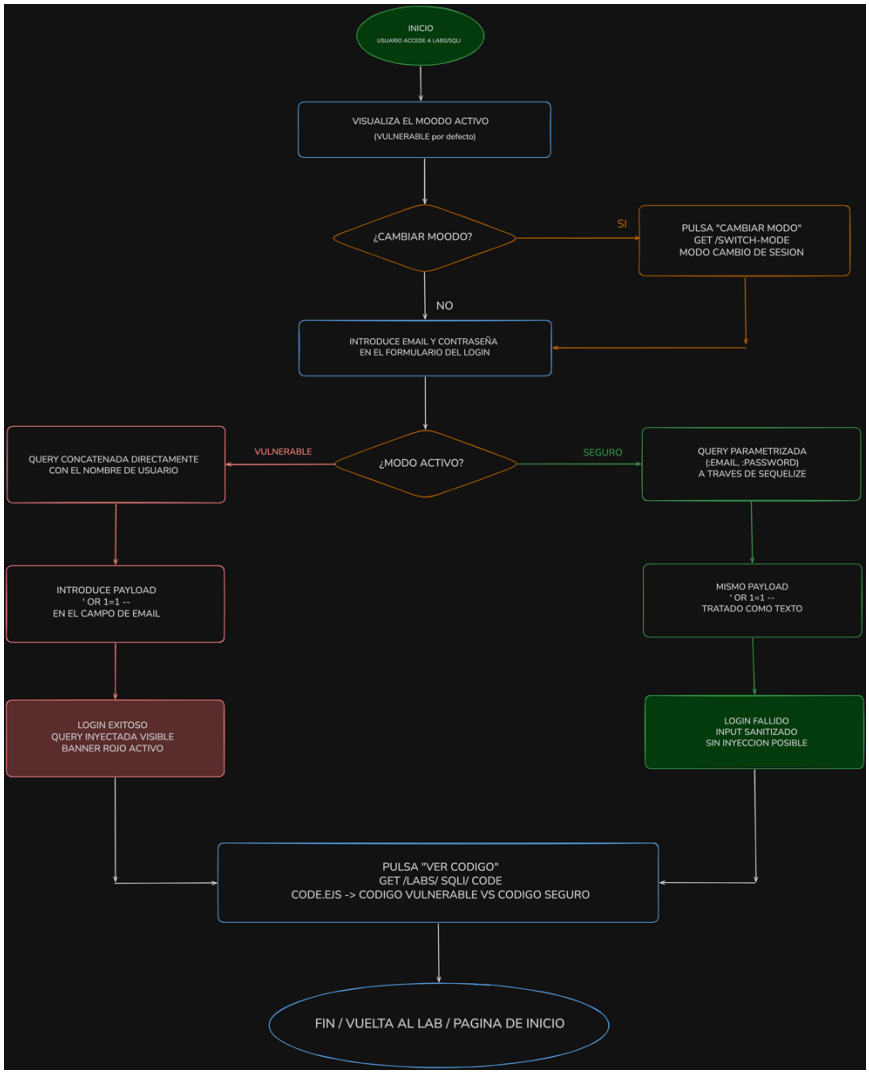

PETICION HTTPS
CLIENTE
RESPUESTA HTML RENDERIZADO
CONTROLADOR (CONTROLLER)
Middleware de Node.js/Express (Globales):
• Helmet
• express-rate-limit
• express-session
• CORS
• body-parser
Modo Vulnerable (Vulnerable)
• Lógica Insegura
• Query Concatenada
• No Sanitización
Modo Seguro (Secure)
• Lógica Protegida
• Query Parametrizada
Sanitización (e.g., replace, bcrypt)
Paso de Datos de Contexto / Renderizado
VISTA (VIEWS)
PLANTILLA BASE (layout.ejs)
• Navbar:
• Inicio
• Labs
• Diccionario
• Compartido pie de página
Vistas de Laboratorios Específicos
Diccionario Interactivo React) Componente React Diccionario para Búsqueda y Filtrado en Tiempo Real
Llamadas a Modelos / Operaciones de Datos
Devolución de Datos
MODELO (MODEL)
Sequelize ORM
Base de Datos PostgreSQL (Relacional)
Almacenamiento de Datos (lab_sqli_users)

7. Casos de uso

A continuación, se describen los casos de uso principales de la plataforma, con su descripción, precondiciones, post-condiciones, datos de entrada y salida, tablas, clases e interfaces implicadas

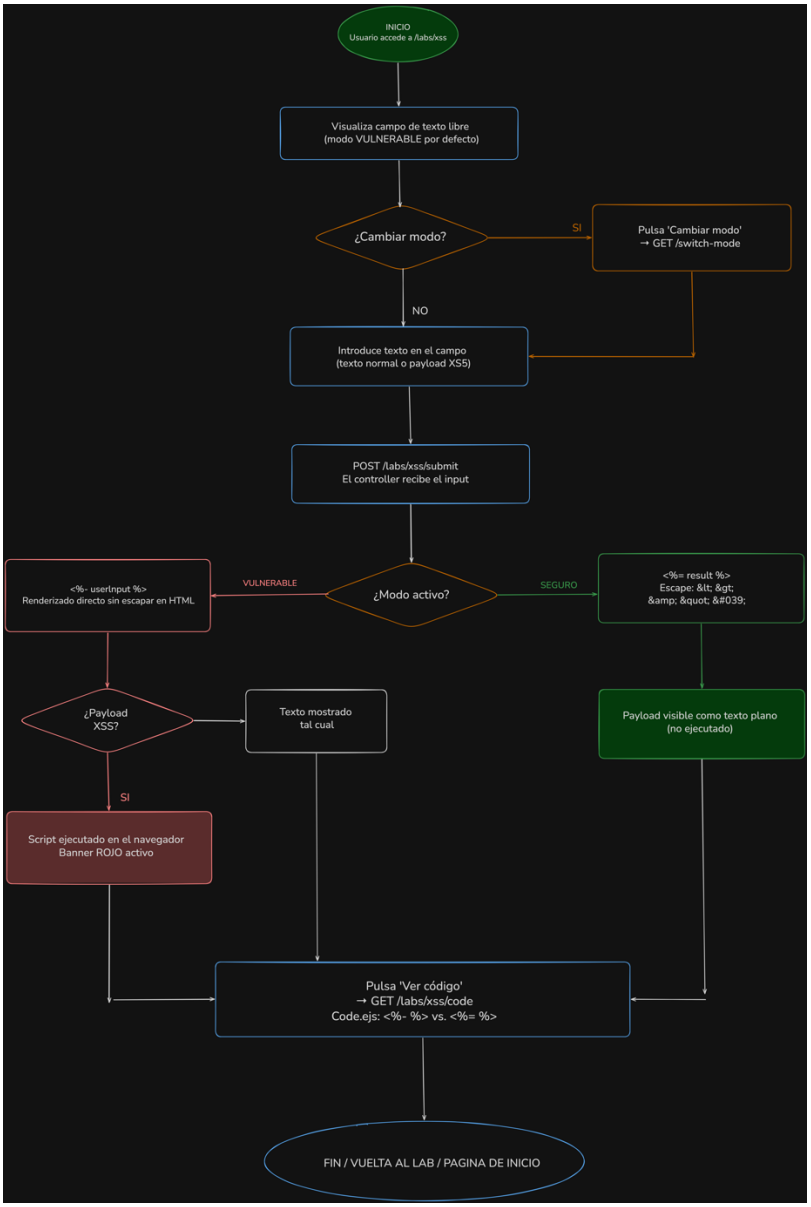
7.1 Caso de uso CU-01: Explotar vulnerabilidad SQL Injection en modo vulnerable

DESCRIPCIÓN: El usuario introduce un payload de SQL Injection en el formulario de login del laboratorio SQLi y obtiene acceso sin credenciales válidas.	
PRECONDICIONES	POSTCONDICIONES
El laboratorio SQLi debe estar en modo VULNERABLE (estado por defecto).	El sistema muestra el mensaje 'Login exitoso (VULNERABLE)' y la query SQL ejecutada con el payload inyectado visible al usuario.
DATOS DE ENTRADA	DATOS DE SALIDA
Email: ' OR 1=1 -- / Password: cualquiera	Query SQL ejecutada. Resultado: Login exitoso. Banner de vulnerabilidad explotada activo.
TABLAS	CLASES
lab_sqli_users	labs/sqli/controller.js → loginSubmit()
INTERFACES	
views/labs/sqli/login.ejs	


INICIO
USUARIO ACCEDE A LABS/SQLI
VISUALIZA EL MOODO ACTIVO
(VULNERABLE por defecto)
¿CAMBIAR MOODO?
SI
PULSA "CAMBIAR MODO"
GET /SWITCH-MODE
MODO CAMBIO DE SESION
NO
INTRODUCE EMAIL Y CONTRASEÑA
EN EL FORMULARIO DEL LOGIN
¿MODO ACTIVO?
VULNERABLE
QUERY CONCATENADA DIRECTAMENTE
CON EL NOMBRE DE USUARIO
SEGURO
QUERY PARAMETRIZADA
(:EMAIL, :PASSWORD)
A TRAVES DE SEQUELIZE
INTRODUCE PAYLOAD
' OR 1=1 --
EN EL CAMPO DE EMAIL
MISMO PAYLOAD
' OR 1=1 --
TRATADO COMO TEXTO
LOGIN EXITOSO
QUERY INYECTADA VISIBLE
BANNER ROJO ACTIVO
LOGIN FALLIDO
INPUT SANITIZADO
SIN INYECCION POSIBLE
PULSA "VER CODIGO"
GET /LABS/ SQLI/ CODE
CODE.EJS -> CODIGO VULNERABLE VS CODIGO SEGURO
FIN / VUELTA AL LAB / PAGINA DE INICIO

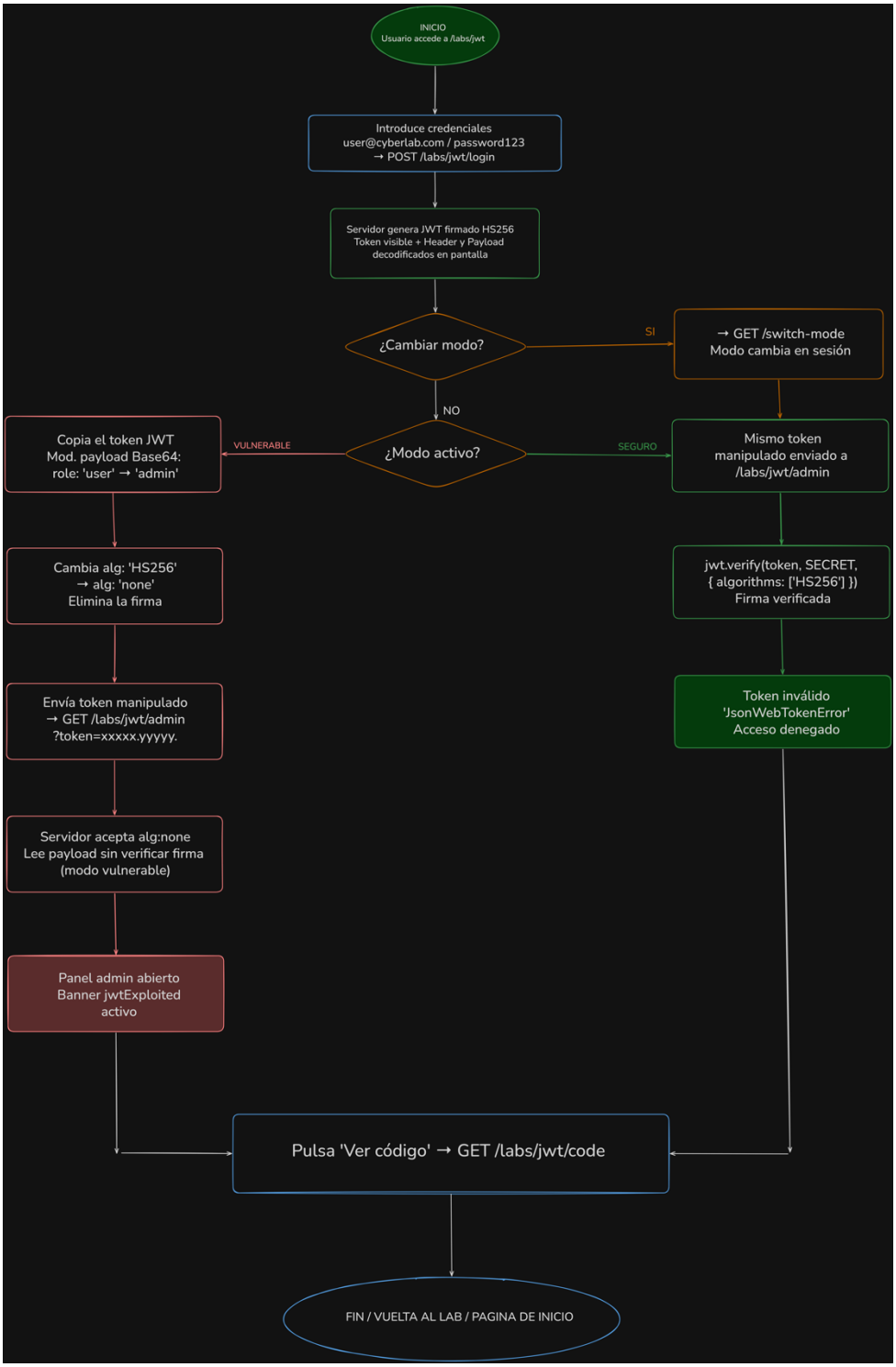
7.2 Caso de uso CU-02: Explotar vulnerabilidad XSS en modo vulnerable

DESCRIPCIÓN: El usuario introduce un payload JavaScript en el campo de texto del laboratorio XSS y consigue que el script se ejecute en el navegador.	
PRECONDICIONES	POSTCONDICIONES
El laboratorio XSS debe estar en modo VULNERABLE.	El payload JavaScript se ejecuta en el navegador. Se muestra el banner de vulnerabilidad explotada.
DATOS DE ENTRADA	DATOS DE SALIDA
Input: <img src=x onerror=alert('XSS')>	Alerta del navegador ejecutada. Modo VULNERABLE activo. Banner de alerta visible.
TABLAS	CLASES
No aplica (sin persistencia)	labs/xss/controller.js → submitForm()
INTERFACES	
views/labs/xss/input.ejs	


INICIO
Usuario accede a /labs/xss
Visualiza campo de texto libre
(modo VULNERABLE por defecto)
¿Cambiar modo?
SI
Pulsa 'Cambiar modo'
→ GET /switch-mode
NO
Introduce texto en el campo
(texto normal o payload XS5)
POST /labs/xss/submit
El controller recibe el input
¿Modo activo?
VULNERABLE
SEGURO
<%- userInput %>
Renderizado directo sin escapar en HTML
<%= result %>
Escape: < >
& " '
¿Payload
XSS?
Texto mostrado
tal cual
Payload visible como texto plano
(no ejecutado)
SI
Script ejecutado en el navegador
Banner ROJO activo
Pulsa 'Ver código'
→ GET /labs/xss/code
Code.ejs: <%- %> vs. <%= %>
FIN / VUELTA AL LAB / PAGINA DE INICIO

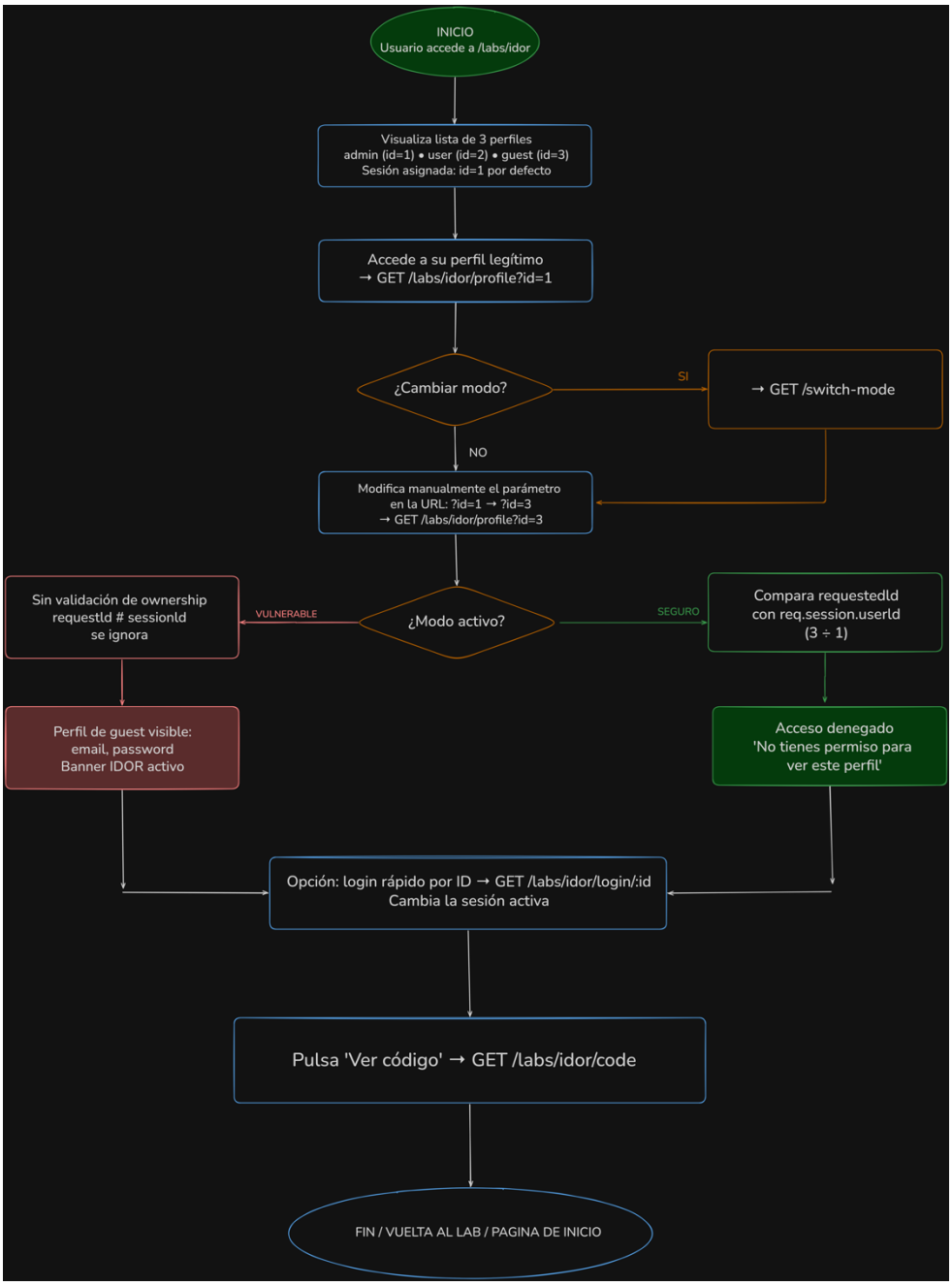
7.3 Caso de uso CU-03: Explotar vulnerabilidad JWT (algoritmo none) en modo vulnerable

DESCRIPCIÓN: El usuario inicia sesión, obtiene un token JWT, lo decodifica y modifica para asignarse el rol 'admin' usando el algoritmo 'none', accediendo al panel de administración.	
PRECONDICIONES	POSTCONDICIONES
Laboratorio JWT en modo VULNERABLE. El usuario ha iniciado sesión con user@cyberlab.com / password123.	El sistema concede acceso al panel de administración y muestra el banner de vulnerabilidad JWT explotada con datos sensibles del sistema.
DATOS DE ENTRADA	DATOS DE SALIDA
Token JWT manipulado: header con alg:'none', payload con role:'admin', sin firma.	Panel de administración accesible. Datos sensibles del sistema visibles. Banner de explotación activo.
TABLAS	CLASES
No aplica (sin persistencia)	labs/jwt/controller.js → admin()
INTERFACES	
views/labs/jwt/local.ejs, views/labs/jwt/admin.ejs	


INICIO
Usuario accede a /labs/jwt
Introduce credenciales
user@cyberlab.com / password123
→ POST /labs/jwt/login
Servidor genera JWT firmado HS256
Token visible + Header y Payload
decodificados en pantalla
¿Cambiar modo?
SI
→ GET /switch-mode
Modo cambia en sesión
NO
¿Modo activo?
VULNERABLE
SEGURO
Copia el token JWT
Mod. payload Base64:
role: 'user' → 'admin'
Mismo token
manipulado enviado a
/labs/jwt/admin
Cambia alg: 'HS256'
→ alg: 'none'
Elimina la firma
jwt.verify(token, SECRET,
{ algorithms: ['HS256'] })
Firma verificada
Envía token manipulado
→ GET /labs/jwt/admin
?token=xxxxx.yyyyy.
Token inválido
'JsonWebTokenError'
Acceso denegado
Servidor acepta alg:none
Lee payload sin verificar firma
(modo vulnerable)
Panel admin abierto
Banner jwtExploited
activo
Pulsa 'Ver código' → GET /labs/jwt/code
FIN / VUELTA AL LAB / PAGINA DE INICIO

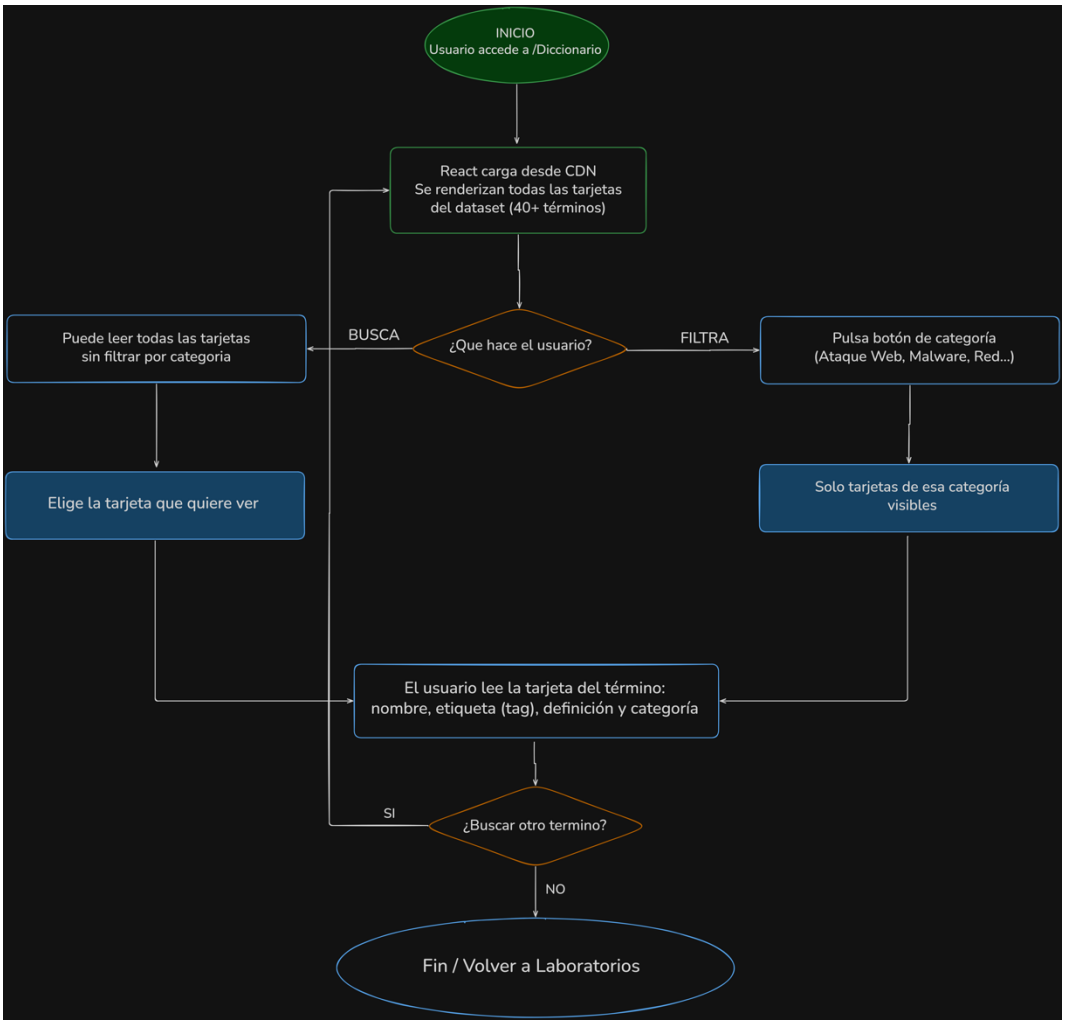
7.4 Caso de uso CU-04: Explotar vulnerabilidad IDOR en modo vulnerable

DESCRIPCIÓN: El usuario autenticado como un perfil modifica el parámetro id en la URL para acceder al perfil de otro usuario y ver sus datos sensibles.	
PRECONDICIONES	POSTCONDICIONES
Laboratorio IDOR en modo VULNERABLE. El usuario ha accedido a su perfil (ej. id=1).	Se muestra el perfil del usuario solicitado (id=3) con todos sus datos, incluyendo contraseña. Banner de IDOR explotado visible.
DATOS DE ENTRADA	DATOS DE SALIDA
URL: /labs/idor/profile?id=3 accedida desde sesión de usuario id=1.	Perfil del usuario guest (id=3): username, email, contraseña en texto plano.
TABLAS	CLASES
No aplica (array en memoria)	labs/idor/controller.js → profilePage()
INTERFACES	
views/labs/idor/index.ejs, views/labs/idor/profile.ejs	


INICIO
Usuario accede a /labs/idor
Visualiza lista de 3 perfiles
admin (id=1) • user (id=2) • guest (id=3)
Sesión asignada: id=1 por defecto
Accede a su perfil legítimo
→ GET /labs/idor/profile?id=1
¿Cambiar modo?
SI
→ GET /switch-mode
NO
Modifica manualmente el parámetro
en la URL: ?id=1 → ?id=3
→ GET /labs/idor/profile?id=3
Sin validación de ownership
requestId # sessionId
se ignora
VULNERABLE
¿Modo activo?
SEGURO
Compara requestedId
con req.session.userId
(3 ÷ 1)
Perfil de guest visible:
email, password
Banner IDOR activo
Acceso denegado
'No tienes permiso para
ver este perfil'
Opción: login rápido por ID → GET /labs/idor/login/:id
Cambia la sesión activa
Pulsa 'Ver código' → GET /labs/idor/code
FIN / VUELTA AL LAB / PAGINA DE INICIO

7.5 Caso de uso CU-05: Buscar términos en el diccionario de ciberseguridad

DESCRIPCIÓN: El usuario accede al diccionario, introduce un término en el buscador y/o selecciona una categoría para filtrar las entradas en tiempo real.	
PRECONDICIONES	POSTCONDICIONES
El usuario ha accedido a la ruta /Diccionario. React ha cargado desde CDN correctamente.	Se muestran en tiempo real únicamente las tarjetas de términos que coinciden con la búsqueda y el filtro de categoría seleccionado.
DATOS DE ENTRADA	DATOS DE SALIDA
Texto de búsqueda (ej: 'JWT') y/o categoría seleccionada (ej: 'Ataque Web').	Lista filtrada de tarjetas con término, etiqueta, definición y badge de categoría.
TABLAS	CLASES
No aplica (dataset estático en JS)	Diccionario/diccionario.ejs → componente React Dictionary
INTERFACES	
Diccionario/diccionario.ejs	


INICIO
Usuario accede a /Diccionario
React carga desde CDN
Se renderizan todas las tarjetas
del dataset (40+ términos)
¿Que hace el usuario?
BUSCA
FILTRA
Puede leer todas las tarjetas
sin filtrar por categoria
Pulsa botón de categoría
(Ataque Web, Malware, Red...)
Elige la tarjeta que quiere ver
Solo tarjetas de esa categoría
visibles
El usuario lee la tarjeta del término:
nombre, etiqueta (tag), definición y categoría
¿Buscar otro termino?
SI
NO
Fin / Volver a Laboratorios

8. DISEÑOS

8.1 Diagrama de la base de datos

La aplicación utiliza actualmente una única tabla real en base de datos para el laboratorio de SQL Injection. El resto de laboratorios (IDOR, Broken Auth, JWT) utilizan bases de datos simuladas en memoria mediante arrays JavaScript, lo que simplifica la configuración del entorno de desarrollo y evita dependencias adicionales para laboratorios que no requieren persistencia.

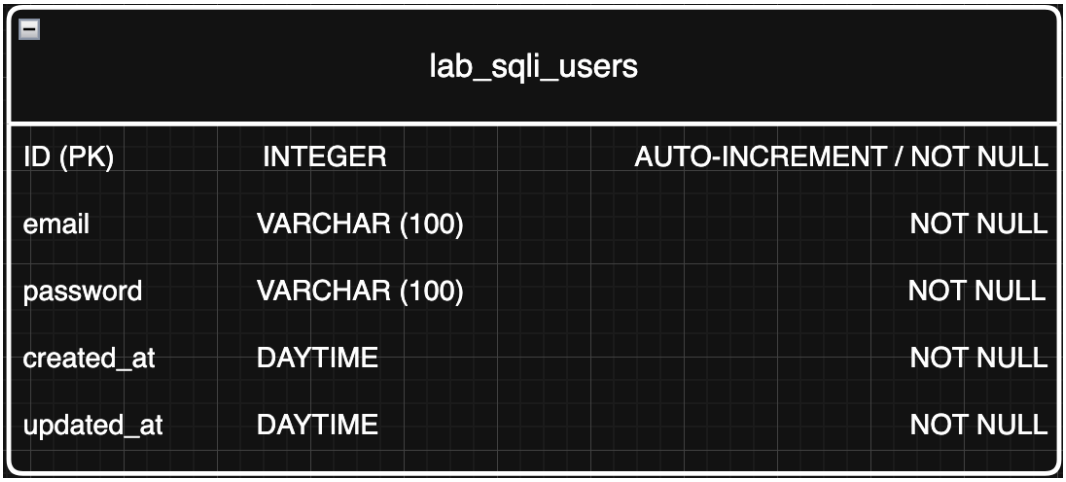
Tabla: lab_sqli_users			
Campo	Tipo	Restricciones	Descripción
id	INTEGER	PK, AUTO_INCREMENT, NOT NULL	Identificador único del usuario.
email	VARCHAR(100)	NOT NULL	Correo electrónico del usuario, usado como credencial de login.
password	VARCHAR(100)	NOT NULL	Contraseña en texto plano (intencionalmente inseguro para el lab).
createdAt	DATETIME	NOT NULL	Fecha y hora de creación del registro. Gestionado automáticamente por Sequelize.
updatedAt	DATETIME	NOT NULL	Fecha y hora de última actualización. Gestionado automáticamente por Sequelize.

Las bases de datos simuladas en memoria de los demás laboratorios mantienen la siguiente estructura:

Laboratorio	Estructura del array en memoria (usersDB)
Broken Authentication	{ username, password, hashedPassword, failedAttempts, locked }
IDOR	{ id, username, password, email, hashedPassword, failedAttempts, locked }
JWT Attacks	{ id, email, password, role }
SSRF	No tiene base de datos. Los flags se devuelven directamente desde los endpoints del router.

8.2 Diagrama E/R (Entidad-Relación)

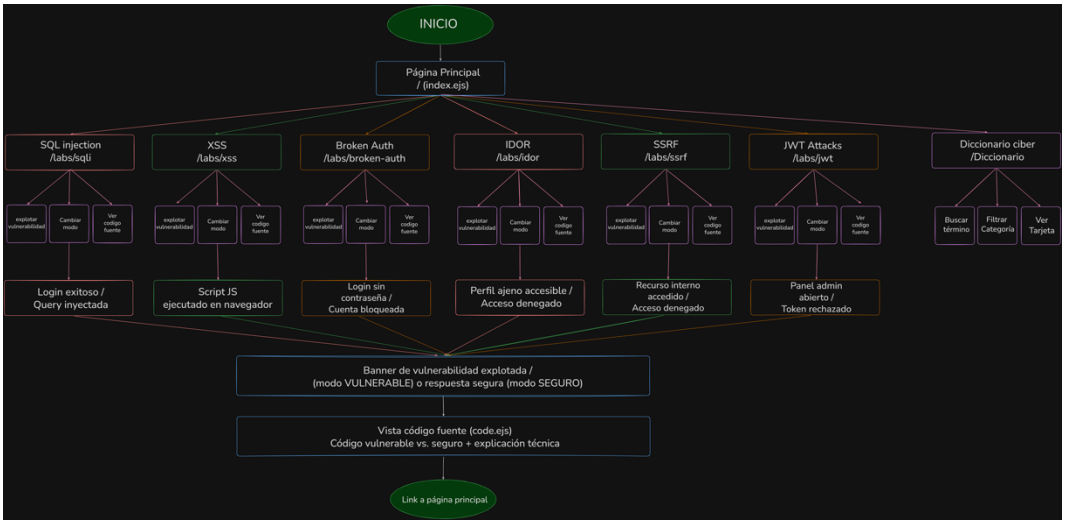
Dado que la mayor parte de los datos de los laboratorios se gestionan en memoria, el modelo de datos persistido es mínimo. La única entidad persistida en base de datos es la tabla lab_sqli_users, utilizada exclusivamente por el laboratorio de SQL Injection.



8.3 Diagrama de flujo de navegación

El siguiente diagrama describe el flujo de navegación del usuario dentro de la plataforma CYBERLAB:

- El usuario accede a la página principal (/) donde encuentra el grid con todos los laboratorios disponibles y el enlace al diccionario.
- Desde la página principal puede navegar al laboratorio de su elección (/labs/sqli, /labs/xss, /labs/broken-auth, /labs/idor, /labs/ssrf, /labs/jwt).
- Dentro de cada laboratorio puede: (a) Explotar la vulnerabilidad en el formulario principal, (b) Cambiar el modo vulnerable/seguro mediante el botón de conmutación, (c) Ver el código fuente y la explicación técnica.
- El diccionario es accesible desde cualquier punto de la plataforma a través del menú de navegación superior.



8.4 Interfaces de usuario

La interfaz de CYBERLAB sigue una temática hacker consistente en todas las vistas: fondo oscuro (#0a0a0a), tipografía monoespaciada (Share Tech Mono / Rajdhani), acento en color verde o azul (según el template heredado del esqueleto), estructura de cuadrícula para la presentación de los laboratorios y alertas visuales diferenciadas para los estados de explotación

Vista / Plantilla EJS	Descripción de la interfaz
views/index.ejs	Página principal con sección hero, presentación de la plataforma y grid de tarjetas de laboratorios disponibles con nombre, descripción, etiqueta de categoría y nivel.
views/layout.ejs	Layout compartido con barra de navegación superior (logo CYBER//LAB, links INICIO, LABORATORIOS, DICCIONARIO) y pie de página con autor y aviso de uso educativo.
views/code.ejs	Vista genérica de código y explicación técnica. Recibe el título, el bloque de código pre-formateado, la explicación completa y el enlace de vuelta al laboratorio.
labs/sqli/login.ejs	Formulario de email/contraseña con tabla de payloads de ejemplo, visualización de la query SQL ejecutada en el último intento y resultado del login.
labs/xss/input.ejs	Campo de texto libre con ejemplos de payloads XSS, área de resultado que renderiza el input directamente (vulnerable) o escapado (seguro).
labs/broken-auth/auth.ejs	Formulario de login con tablas de credenciales de prueba (válidas e inválidas), resultado del intento, explicación técnica y estado de bloqueo de cuenta.
labs/idor/index.ejs	Lista de perfiles disponibles con botones de acceso directo por ID y explicación del vector de ataque IDOR.

labs/idor/profile.ejs	Vista de perfil de usuario que muestra todos los datos del usuario (incluyendo contraseña en modo vulnerable) con banner de explotación si se accedió a un perfil ajeno.
labs/ssrf/local.ejs	Formulario de URL con tabla de URLs de ejemplo y sus efectos, área de resultado de la petición al servidor y banner de explotación SSRF.
labs/jwt/local.ejs	Formulario de login JWT con visualización del token generado, header decodificado y payload decodificado en tiempo real.
labs/jwt/admin.ejs	Panel de administración que muestra datos sensibles del sistema simulado, accesible solo con rol admin (legítimo o mediante explotación JWT).
Diccionario/diccionario.ejs	Diccionario interactivo con buscador, botones de filtro por categoría generados dinámicamente y grid de tarjetas de términos. Implementado con React.

Imágenes de las diferentes vistas de la aplicación en varios dispositivos:

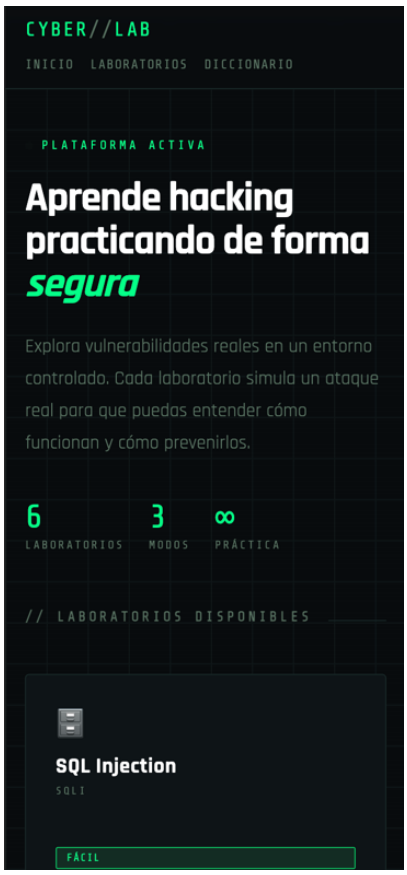
- Imagen de la pagina principal en versión escritorio 1728 x 1117:

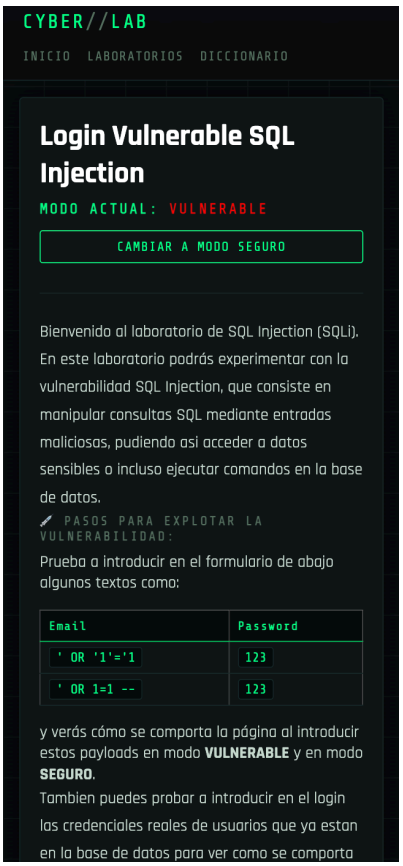


- Imagen de la pagina principal en versión Tablet 1194 x 976:



- Imagen de la pagina principal versión movil 390 x 844:

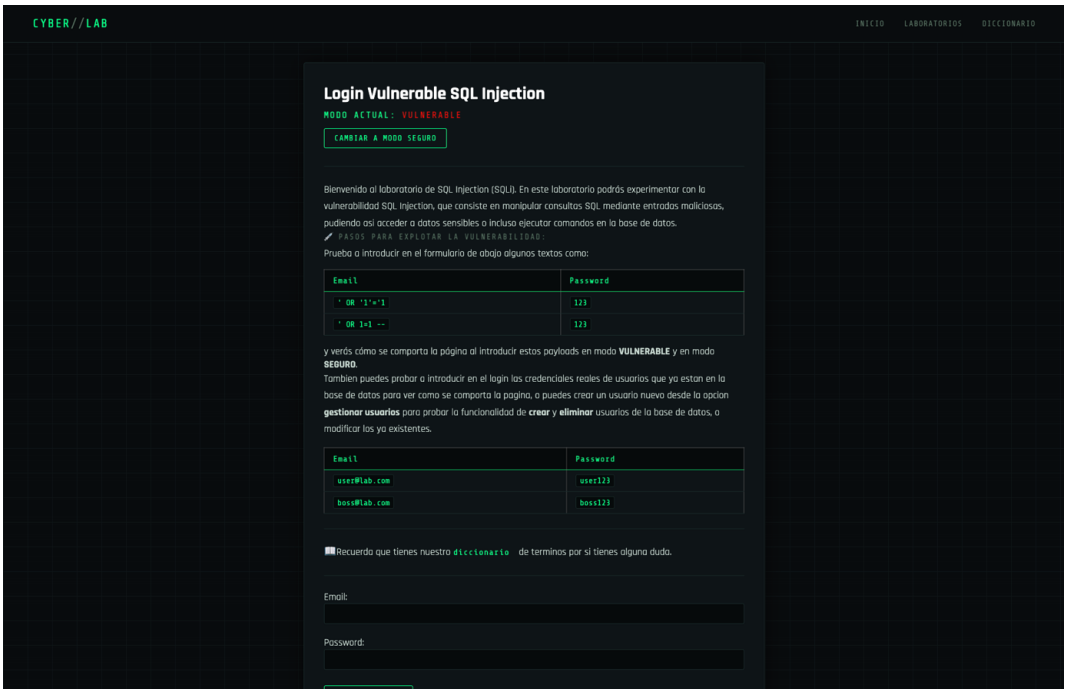




9. Descripción detallada de los laboratorios

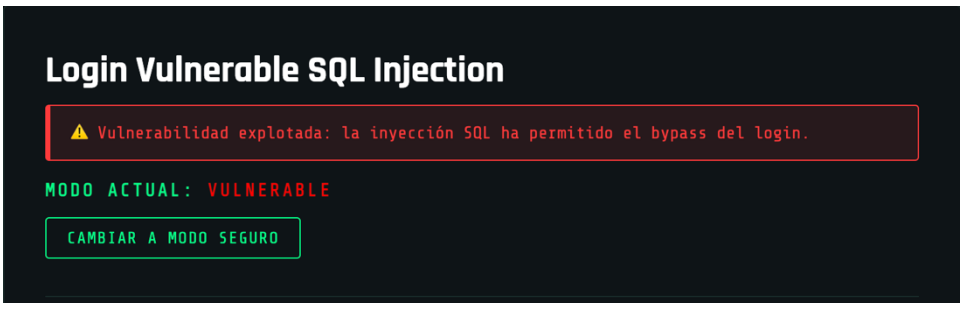
9.1 Laboratorio SQL Injection (SQLi)

El laboratorio de SQL Injection simula un formulario de login que, en modo vulnerable, construye la consulta SQL concatenando directamente los valores introducidos por el usuario en la cadena de texto de la query. Esto permite a un atacante inyectar sentencias SQL arbitrarias para eludir la autenticación o extraer datos de la base de datos sin disponer de credenciales válidas.



En modo vulnerable, la query ejecutada por el servidor es la siguiente:

```
SELECT * FROM lab_sqli_users WHERE email =
'[VALOR_INTRODUCIDO_POR_EL_USUARIO]'
```



Un atacante puede explotar esta vulnerabilidad introduciendo payloads como los siguientes en el campo de email:

- ' OR '1'='1 → La condición OR '1'='1' es siempre verdadera, por lo que la query devuelve todos los registros de la tabla.

- ' OR 1=1 -- → El doble guion comenta el resto de la query, ignorando cualquier condición adicional como la verificación de contraseña.



En modo seguro, la consulta utiliza parámetros reemplazados de Sequelize, lo que impide que el motor SQL interprete el input del usuario como código SQL:

```
SELECT * FROM lab_sqli_users WHERE email = :email AND password = :password
```



Con parámetros reemplazados, el input ' OR 1=1 -- es tratado literalmente como una cadena de texto y no como código SQL, por lo que la búsqueda no devuelve ningún resultado y el login falla correctamente. La vista del laboratorio muestra siempre la query ejecutada en el último intento, lo que permite al usuario comparar visualmente ambos comportamientos.

Este lab incorpora también una sección de gestión de usuarios accesible desde la ruta */labs/sqli/users,* cuyo propósito es permitir al usuario de la plataforma administrar directamente los registros de la tabla que usa el laboratorio durante la práctica. Esta funcionalidad sirve como herramienta auxiliar del laboratorio: el alumnoo puede crear nuevas cuentas para probar distintos payloads de inyección, modificar las credenciales de los registros existentes o eliminar entradas que ya no necesite, todo ello sin necesidad de acceder diretamente a la base de datos.



La vista asociada desde *labs/sqli/users.ejs* presenta la interfaz dividida en dos bloques diferenciados. El primero contiene un formulario de creación con dos campos de texto libres, email y password, que envía una petición POST a *labs/sqli/users/créate.* El segundo bloque muestra la lista completa de usuarios recuperados de la base de datos, ordenados por ID ascendente, que incluye también los campos *email* y *password* en texto plano, de forma intencionadamente insegura para reforzar el objetivo didáctico del laboratorio, Cada entrada de la tabla incluye un botón de actualizar para modificar los campos del registro y otro botón de borrar por si se quiere eliminar esa entrada.



Las cuatro operaciones están implementadas en *controller.js* e intercomunicadas con la base de datos a través de Sequelize con consultas parametrizadas mediante replacements, garantizando que la propia interfaz de gestión no sea en sí misma vulnerable a inyección SQL, a diferencia del formulario de login principal del laboratorio:

La operación de lectura (usersPage) ejecuta un SELECT * FROM lab_sqli_users ORDER BY id ASC y pasa el array de resultados a la vista junto con los mensajes de éxito o error propagados como parámetros de query string (?success= / ?error=).

La operación de creación (createUser) valida que ambos campos estén presentes antes de ejecutar un INSERT INTO lab_sqli_users (email, password) VALUES (:email, :password). En caso de que el email ya exista —la columna tiene restricción de unicidad— el error de base de datos es capturado y comunicado al usuario mediante redirección con mensaje de error en la query string.

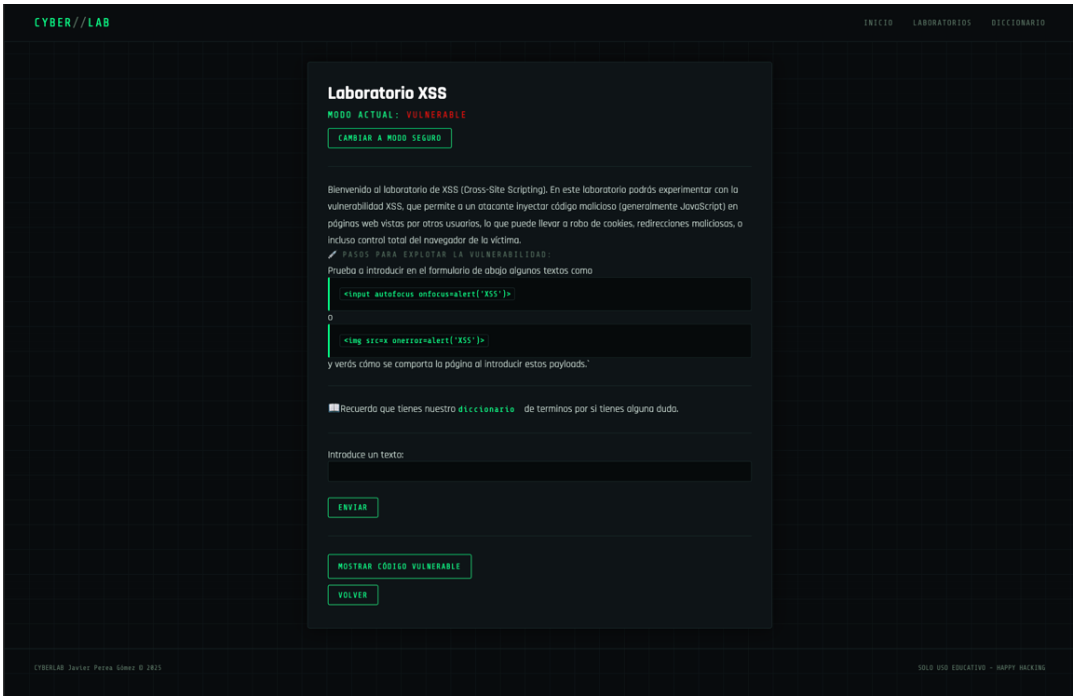
La operación de actualización (updateUser) recibe el id del usuario como parámetro de ruta *(/users/update/:id)* junto con los nuevos valores de email y password en el cuerpo de la petición, y ejecuta un UPDATE lab_sqli_users SET email = :email, password = :password WHERE id = :id. Al igual que en el resto de operaciones, la validación de presencia de campos se realiza antes de consultar la base de datos.

La operación de eliminación (deleteUser) recibe el id por parámetro de ruta *(/users/delete/:id)* y ejecuta un DELETE FROM lab_sqli_users WHERE id = :id, redirigiendo posteriormente al listado con el mensaje de confirmación correspondiente.

El resultado de cada operación se comunica siempre mediante redirección a */labs/sqli/users* con un parámetro ?success= o ?error= en la URL, que la vista recoge y renderiza como un banner de confirmación o de error en la parte superior de la página.

9.2 Laboratorio Cross-Site Scripting (XSS)

El laboratorio de XSS muestra cómo un campo de texto libre puede convertirse en un vector de inyección de código JavaScript malicioso cuando la aplicación no sanitiza el input del usuario antes de incluirlo en el HTML de la respuesta. En modo vulnerable, el valor introducido por el usuario se renderiza directamente en el HTML mediante la etiqueta EJS no escapada <%- %>, que inserta el contenido tal cual en el DOM.

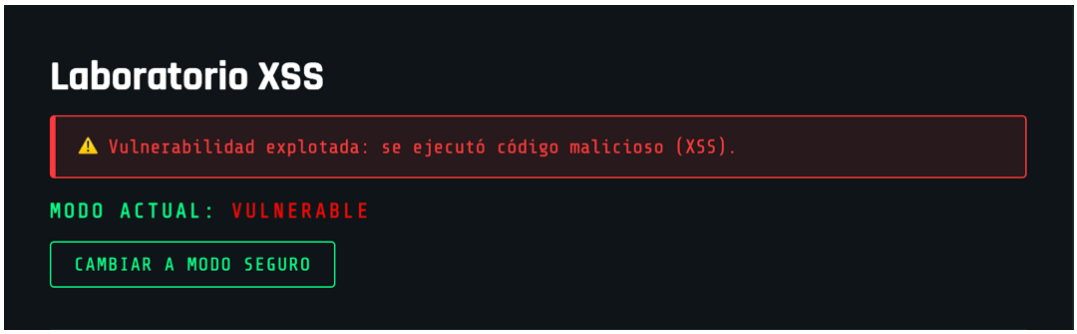


Payloads de explotación típicos que el usuario puede probar en el formulario:

```
<script>alert("XSS")</script>

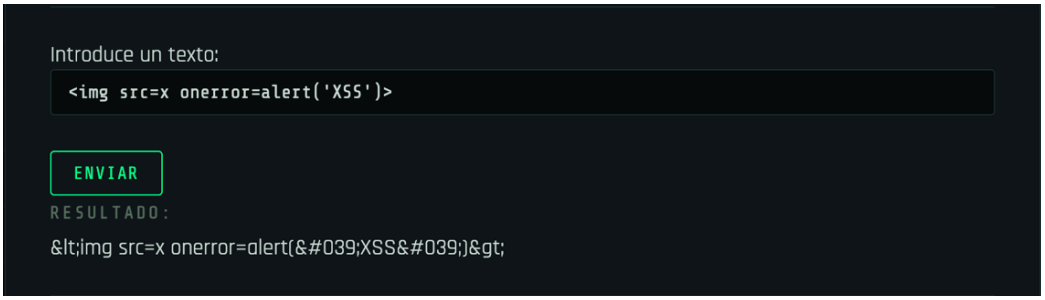
<img src=x onerror=alert("XSS")>

<input autofocus onfocus=alert('XSS')>
```



En modo seguro, el controller aplica un proceso de escape de caracteres especiales HTML sobre el input antes de enviarlo a la vista, y la plantilla utiliza la etiqueta EJS segura <%= %> que también escapa el contenido:

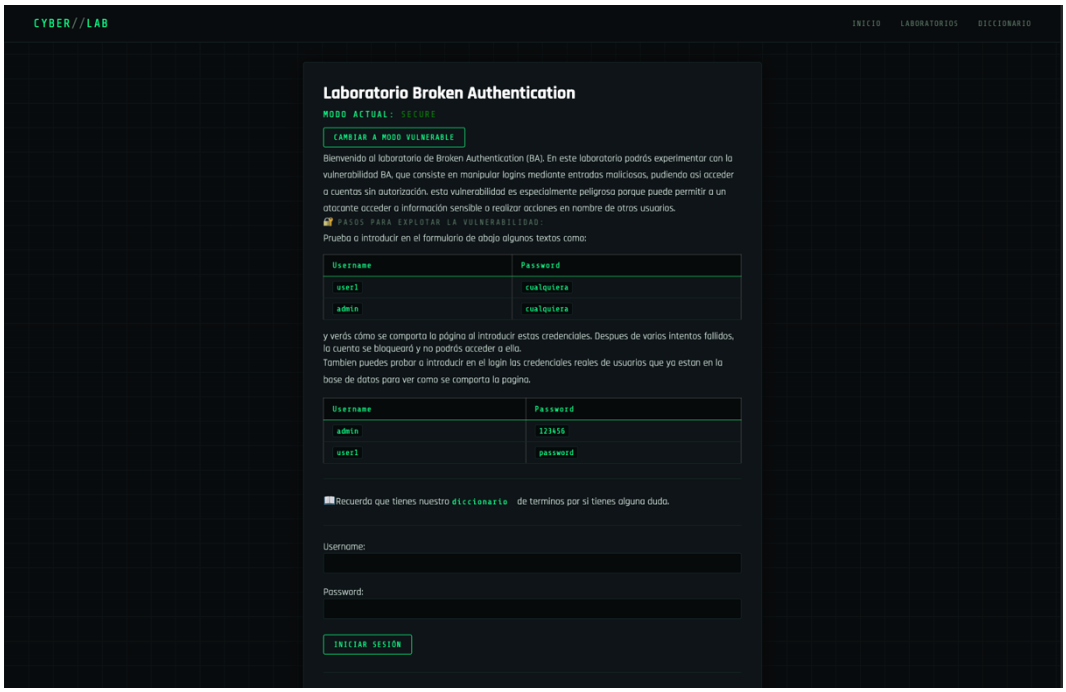
```
result = userInput.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;').replace(/"/g, '&quot;').replace(/'/g, '&#039;');
```



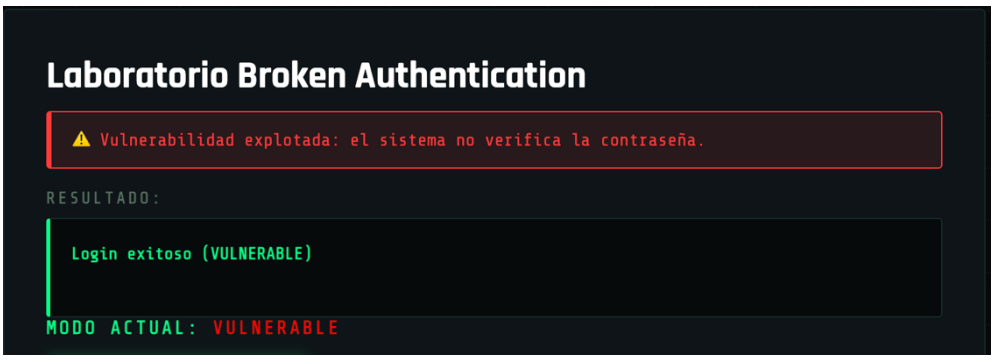
El laboratorio incluye una función isXSS() que detecta automáticamente si el payload contiene patrones XSS comunes (etiquetas script, eventos inline como onerror, onfocus, onmouseover, o el prefijo javascript:) y activa un banner de alerta cuando la vulnerabilidad ha sido explotada con éxito en modo vulnerable.

9.3 Laboratorio Broken Authentication

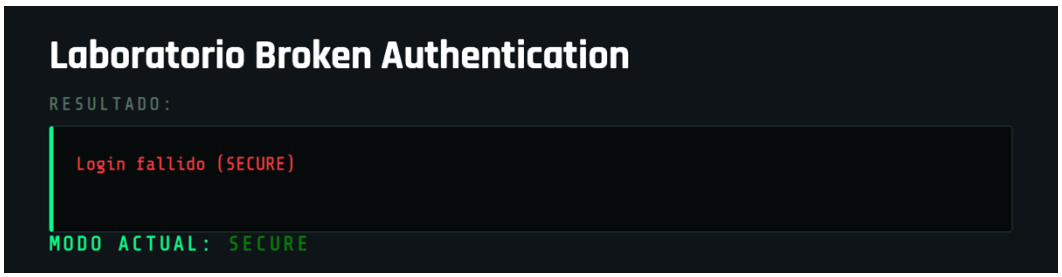
El laboratorio **Broken Authentication** demuestra cómo una implementación incorrecta del proceso de autenticación permite acceder a cuentas sin verificar correctamente las credenciales del usuario. El sistema utiliza una base de datos simulada en memoria (usersDB) que contiene usuarios con atributos como *username*, *password*, *hashedPassword*, *failedAttempts* y *locked*.



En modo vulnerable, el sistema no valida adecuadamente la contraseña introducida. El controller permite el acceso estableciendo directamente el resultado de autenticación como exitoso, sin comprobar si la contraseña coincide con la del usuario. Esto permite a cualquier usuario autenticarse con cualquier cuenta simplemente proporcionando un nombre de usuario válido. La aplicación detecta este comportamiento inseguro y activa un indicador de autenticación vulnerable.



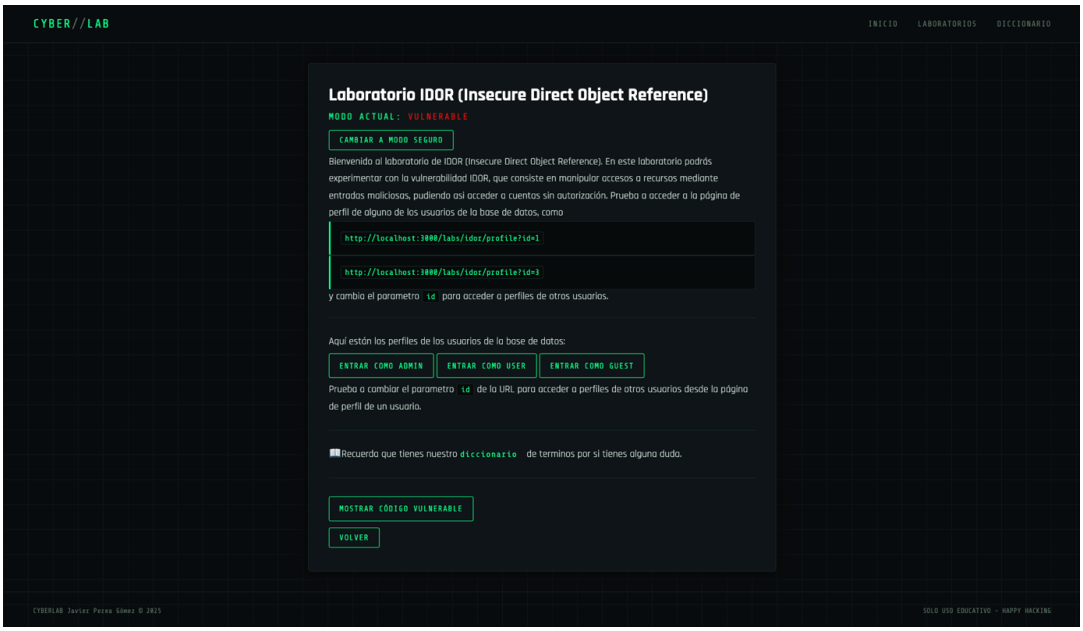
En modo seguro, el sistema utiliza la librería **bcrypt** para comparar la contraseña introducida con el hash almacenado (bcrypt.compare). Además, implementa medidas de protección adicionales como el conteo de intentos fallidos (*failedAttempts*) y el bloqueo de la cuenta (*locked*) tras múltiples intentos incorrectos. De este modo, solo se permite el acceso cuando las credenciales son válidas, evitando accesos no autorizados y ataques de fuerza bruta.



La lógica de control se basa en validar correctamente las credenciales mediante funciones de hash seguras y en gestionar el estado de la cuenta del usuario dentro de la sesión, garantizando un proceso de autenticación robusto.

9.4 Laboratorio IDOR (Insecure Direct Object Reference)

El laboratorio IDOR demuestra cómo el uso de referencias directas a objetos controlables por el usuario (el parámetro id en la URL) sin validación de permisos permite acceder a recursos de otros usuarios. La base de datos simulada contiene tres perfiles: admin (id=1), user (id=2) y guest (id=3).

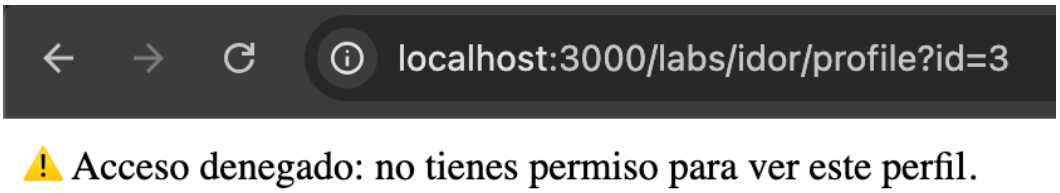


En modo vulnerable, cualquier usuario puede acceder al perfil de cualquier otro simplemente modificando el parámetro ?id= en la URL. La vista de perfil expone datos sensibles como el nombre de usuario, el correo electrónico y la contraseña en texto plano. El sistema detecta si el id solicitado difiere del id de la sesión activa y activa el banner de explotación IDOR.



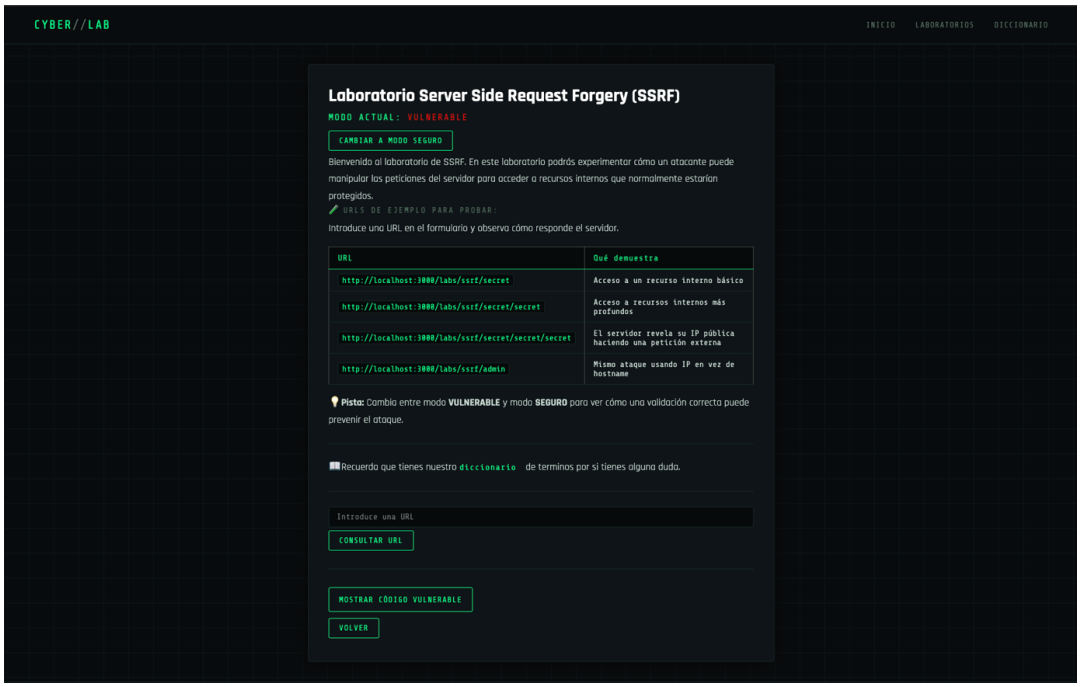


En modo seguro, el controller compara el id solicitado con el id de la sesión activa y devuelve un mensaje de 'Acceso denegado' si no coinciden, impidiendo el acceso a perfiles ajenos. La lógica de detección se basa en comparar el req.session.lastVisitedId con el requestedId de la petición actual.



9.5 Laboratorio SSRF (Server-Side Request Forgery)

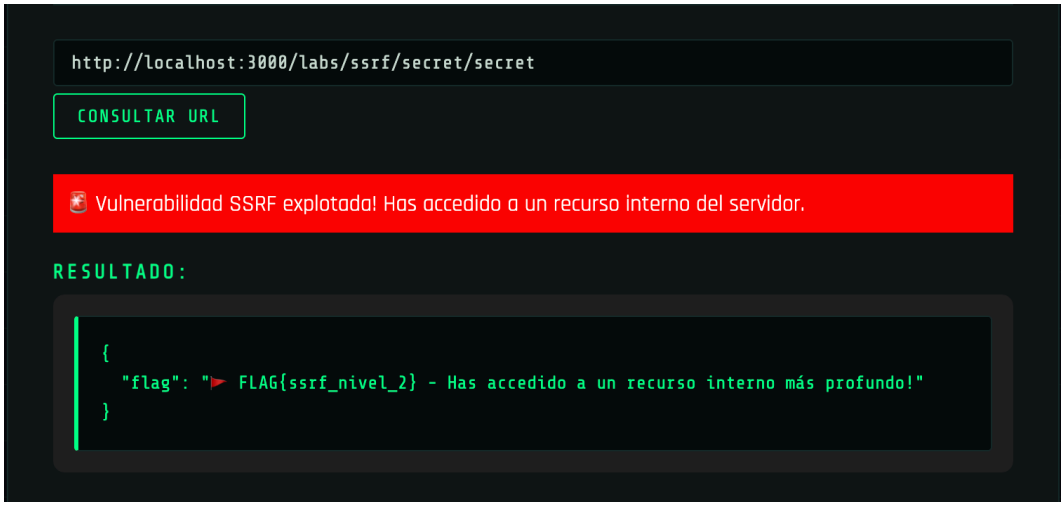
El laboratorio SSRF presenta un formulario donde el usuario puede introducir una URL arbitraria que el servidor consultará usando axios. En modo vulnerable, el servidor realiza la petición sin ninguna validación, lo que permite al atacante hacer que el servidor acceda a recursos internos normalmente inaccesibles desde el exterior.



Los endpoints internos con flags disponibles para descubrir mediante el formulario SSRF son:

- GET /labs/ssrf/secret → { flag: 'FLAG{ssrf_nivel_1} - Has accedido al recurso interno básico!' }
- GET /labs/ssrf/secret/secret → { flag: 'FLAG{ssrf_nivel_2} - Has accedido a un recurso interno más profundo!' }

- GET /labs/ssrf/secret/secret/secret → { flag: 'FLAG{ssrf_nivel_3} - Has comprometido por completo el servidor!' }
- GET /labs/ssrf/admin → { flag: 'FLAG{ssrf_nivel_4} - Has accedido al panel de administración interno!' }

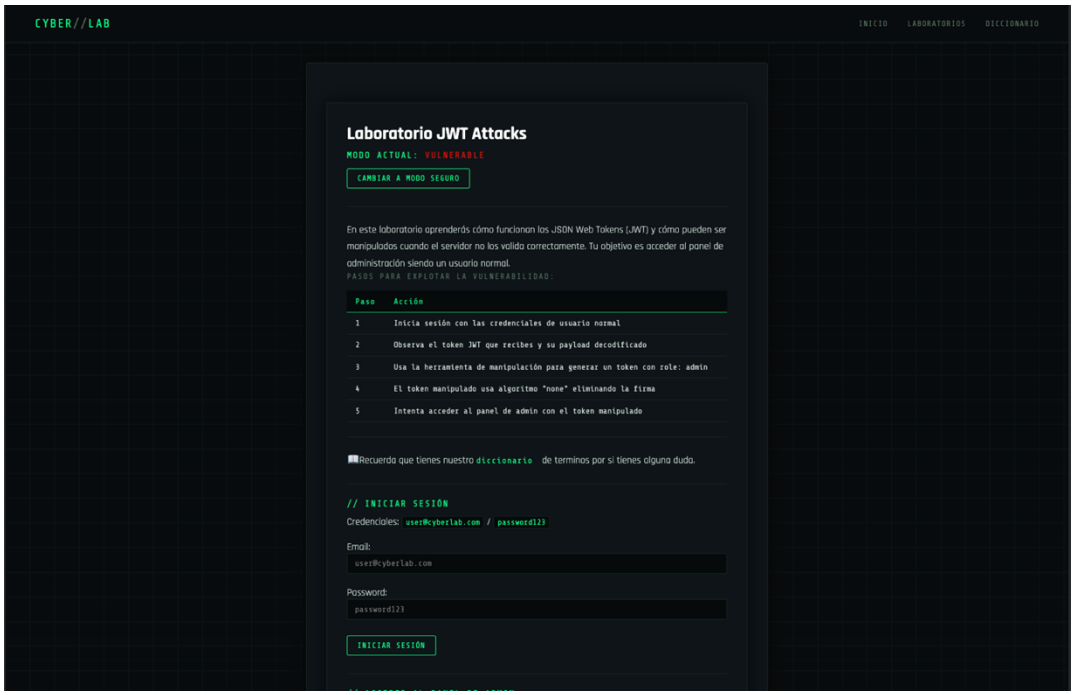


En modo seguro, el controller aplica una blacklist de términos sospechosos antes de realizar la petición, bloqueando cualquier URL que contenga: localhost, 127.0.0.1, 0.0.0.0, internal, admin o /secret. La respuesta al usuario incluye el contenido de la petición (truncado a 500 caracteres) junto con el código de estado HTTP y el tipo de contenido recibido.



9.6 Laboratorio JWT Attacks

El laboratorio de JWT Attacks ilustra los fallos más comunes en la implementación de JSON Web Tokens. Un JWT está compuesto por tres partes codificadas en Base64 separadas por puntos: cabecera (header) con el algoritmo de firma, payload con los datos del usuario (id, email, rol) y firma (signature) que garantiza la integridad del token.



El usuario de prueba disponible para el login es user@cyberlab.com con contraseña password123. Al iniciar sesión, el servidor genera un token JWT firmado con el algoritmo HS256 y la clave secreta 'supersecret123' (intencionalmente débil para el laboratorio), con un payload que incluye id, email y role: 'user'.

El flujo de explotación en modo vulnerable consta de los siguientes pasos:

- 1. Iniciar sesión con las credenciales correctas.



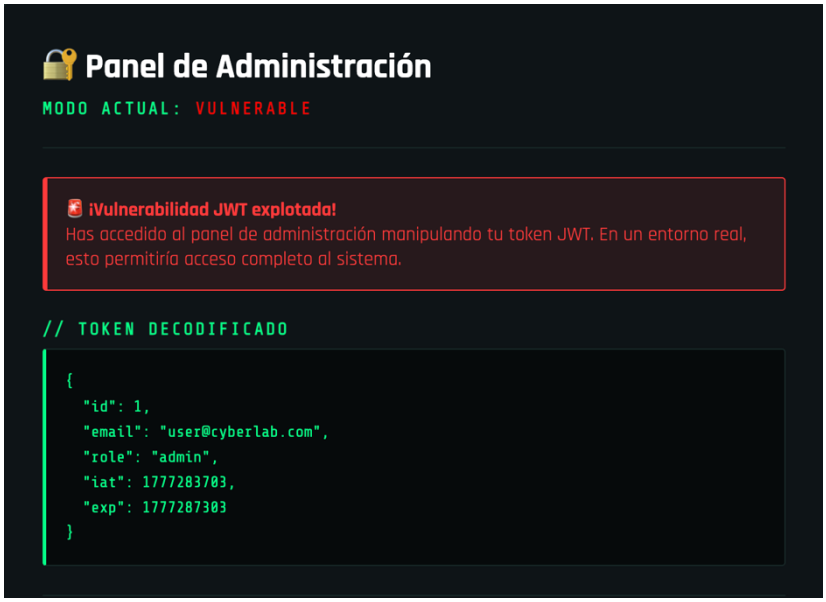
- 2. Observar el token generado y el payload recibido.



- 3. Se usa la herramienta de manipulación de token para modificar el rol de “user” a “admin”, eliminando la firma del token original.



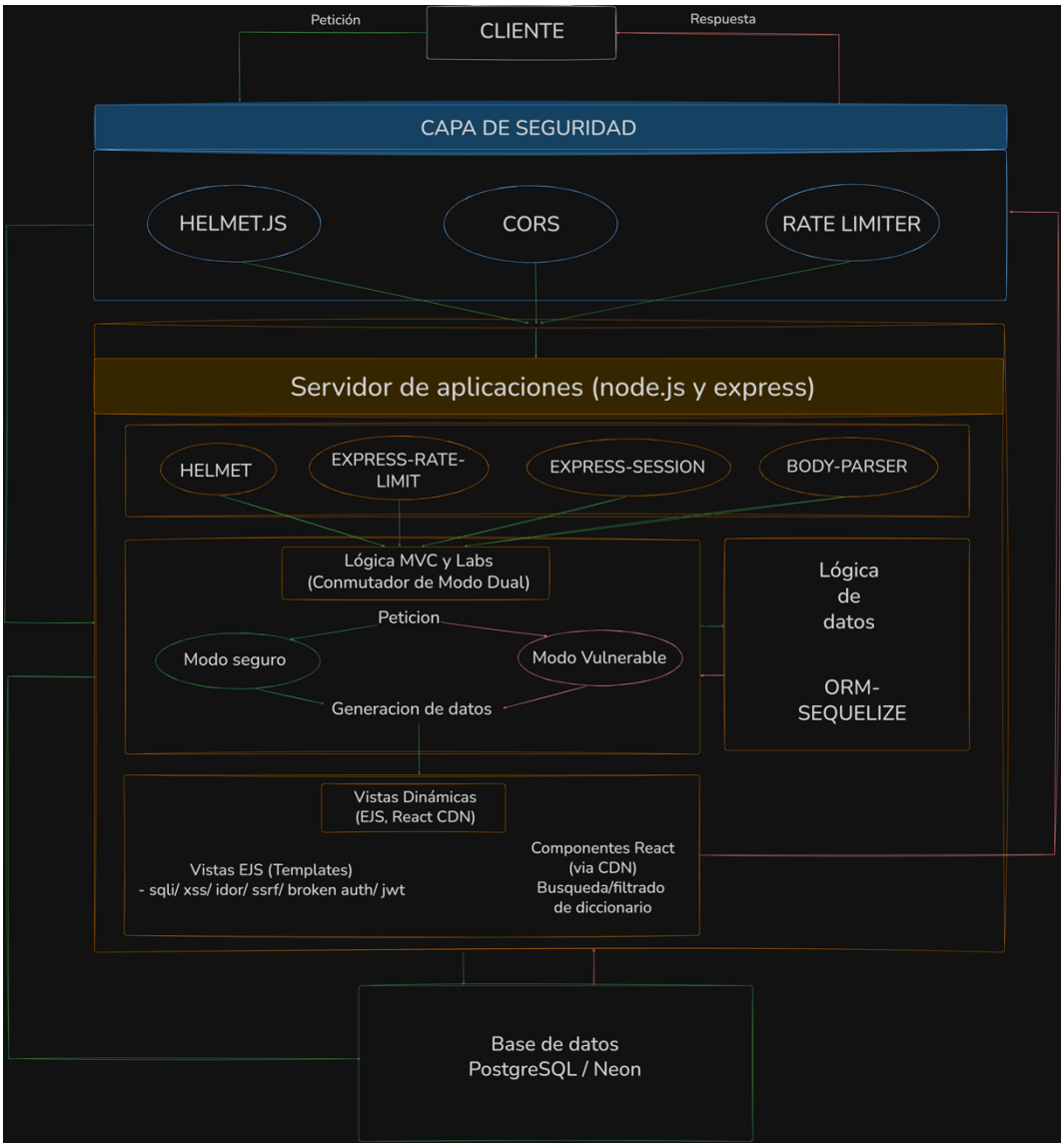
- 4. Enviar el token manipulado al endpoint /labs/jwt/admin y obtener acceso al panel de administración.



En modo seguro, el servidor fuerza el uso del algoritmo HS256 mediante la opción {{ algorithms: ['HS256'] }} de la función jwt.verify(), lo que impide aceptar tokens con algoritmo 'none' o cualquier otro no autorizado. La firma del token es verificada criptográficamente antes de leer el payload, por lo que cualquier modificación del payload invalida la firma y el token es rechazado.



9.7 Diagrama de red



10. TECNOLOGÍAS UTILIZADAS

A continuación, se detallan las tecnologías y herramientas utilizadas en el desarrollo de CYBERLAB, junto con la justificación de su elección y el uso concreto que se hace de cada una en el proyecto.

Tecnología / Herramienta	Versión aprox.	Descripción y uso en el proyecto
Node.js	v18+	Entorno de ejecución JavaScript del lado del servidor. Elegido por su ecosistema de paquetes (npm), rendimiento en aplicaciones I/O intensivas y la familiaridad del alumno con JavaScript tanto en frontend como en backend.
Express.js	4.x	Framework web minimalista sobre Node.js que gestiona el enrutamiento HTTP y la cadena de middlewares. Utilizado para registrar los routers de cada laboratorio y configurar los middlewares globales de seguridad.
EJS (Embedded JavaScript)	3.x	Motor de plantillas que genera HTML dinámico con JavaScript embebido. Permite diferenciar entre renderizado seguro (<%= %>) y renderizado sin escape (<%- %>) para demostrar la vulnerabilidad XSS.
express-ejs-layouts	2.x	Middleware que añade soporte para layouts compartidos en EJS. Permite definir un layout base (layout.ejs) con navegación y footer que es heredado automáticamente por el resto de vistas.
Sequelize	6.x	ORM para Node.js compatible con MySQL, PostgreSQL y SQLite. Utilizado para la definición del modelo de la tabla lab_sqli_users, la sincronización del esquema y la ejecución de consultas parametrizadas en el lab SQLi.
bcrypt	5.x	Librería de hashing de contraseñas con factor de coste configurable. Utilizada en el laboratorio de Broken Authentication para implementar el almacenamiento seguro de contraseñas y la verificación mediante bcrypt.compare().

jsonwebtoken (JWT)	9.x	Implementación del estándar RFC 7519 para Node.js. Utilizada en el laboratorio de JWT Attacks para generar tokens firmados con HS256, decodificarlos y demostrar la vulnerabilidad del algoritmo 'none'.
axios	1.x	Cliente HTTP para Node.js y el navegador. Utilizado en el laboratorio SSRF para realizar las peticiones HTTP desde el backend del servidor a las URLs proporcionadas por el usuario, simulando el vector de ataque real.
Helmet	7.x	Middleware de seguridad que añade múltiples cabeceras HTTP de protección (X-Content-Type-Options, X-Frame-Options, Referrer-Policy, etc.). Aplicado globalmente en app.js con CSP desactivado intencionalmente para no interferir con los laboratorios.
express-rate-limit	7.x	Middleware de limitación de tasa para Express. Configurado globalmente con ventana de 15 minutos y máximo de 500 peticiones por IP, devolviendo HTTP 429 al superarlo.
express-session	1.x	Middleware de gestión de sesiones HTTP con almacenamiento en memoria del servidor. Utilizado para mantener el modo activo (vulnerable/seguro) y el usuario simulado de cada laboratorio entre peticiones.
CORS (cors)	2.x	Middleware para gestión de cabeceras Cross-Origin Resource Sharing. Habilitado globalmente para permitir futuras integraciones con frontends externos o APIs de terceros.
dotenv	16.x	Gestión de variables de entorno desde un fichero .env para no exponer credenciales en el código fuente. Utilizado para la configuración del puerto del servidor y los parámetros de conexión a la base de datos.
React (CDN)	18.x	Librería de UI de Meta cargada desde CDN de Cloudflare (react.production.min.js y react-dom.production.min.js). Utilizada exclusivamente en la vista del diccionario para implementar el componente interactivo de búsqueda y filtrado sin necesidad de bundler ni compilación.
CSS personalizado (style.css)	—	Hoja de estilos global escrita a mano con variables CSS, tipografía Share Tech Mono / Rajdhani (Google Fonts),

		grid responsivo de laboratorios, tarjetas, alertas de vulnerabilidad, botones y temática dark hacker.
Git / GitHub	—	Sistema de control de versiones distribuido. El repositorio está alojado en https://github.com/Javip1991/Proyecto_TFG_LAB para el seguimiento de cambios, historial de commits y posible colaboración futura.
Visual Studio Code	—	Entorno de desarrollo integrado (IDE) principal utilizado durante el desarrollo, con extensiones para Node.js, EJS, JavaScript y control de versiones Git integrado.
Postman / curl	—	Herramientas auxiliares utilizadas durante el desarrollo para probar las rutas HTTP de los laboratorios, verificar las respuestas del servidor y depurar la lógica de los controllers.

11. METODOLOGÍA

11.1 Metodología de desarrollo

El proyecto ha sido desarrollado siguiendo una metodología ágil adaptada para un proyecto individual, inspirada en los principios de Scrum pero sin los ceremoniales formales propios de un equipo. El desarrollo se organizó en iteraciones cortas de una a dos semanas, cada una centrada en la entrega de un laboratorio funcional completo, desde el análisis de la vulnerabilidad hasta las pruebas de verificación.

Esta aproximación iterativa e incremental se justifica por las características del proyecto: la naturaleza modular de los laboratorios permite desarrollarlos de forma independiente, con entregables demostrables al final de cada iteración. Esto ha facilitado las revisiones periódicas con el tutor y la detección temprana de problemas de diseño o implementación antes de que afectaran al resto del proyecto.

Cada iteración seguía el ciclo: (1) análisis de la vulnerabilidad y diseño del laboratorio, (2) implementación del modo vulnerable con pruebas de explotación, (3) implementación del modo seguro con pruebas de mitigación, (4) creación de la vista EJS y la vista de código, (5) integración con el layout compartido y pruebas de regresión sobre los laboratorios anteriores.

11.2 Planificación inicial estimada (antes de la implementación)

Fase / Tarea	Horas estimadas
Análisis y diseño de la arquitectura MVC	8 h
Configuración del entorno (Node.js, Express, EJS, Sequelize, dotenv)	6 h
Desarrollo del layout compartido y página principal	6 h
Laboratorio SQL Injection (controller, router, vista, BD)	8 h
Laboratorio XSS (controller, router, vista)	8 h
Laboratorio Broken Authentication (controller, router, vista, bcrypt)	6 h
Laboratorio IDOR (controller, router, vistas de lista y perfil)	8 h
Laboratorio SSRF (controller, router, vista, endpoints con flags)	6 h
Laboratorio JWT Attacks (controller, router, vistas de login y admin)	12 h
Diccionario de ciberseguridad (dataset, componente React, vista EJS)	6 h
Estilos CSS globales (temática, grid, tarjetas, alertas, responsive)	10 h
Configuración de seguridad de la plataforma (Helmet, CORS, rate limiting)	4 h
Pruebas funcionales de todos los laboratorios y depuración	10 h
Redacción de la memoria del TFG	20 h
TOTAL ESTIMADO	118 h

11.3 Planificación real ejecutada (tiempo dedicado finalmente)

Fase / Tarea	Horas reales
Análisis y diseño de la arquitectura MVC	10 h
Configuración del entorno (Node.js, Express, EJS, Sequelize, dotenv)	8 h
Desarrollo del layout compartido y página principal	7 h
Laboratorio SQL Injection (controller, router, vista, BD)	12 h
Laboratorio XSS (controller, router, vista)	8 h
Laboratorio Broken Authentication (controller, router, vista, bcrypt)	8 h
Laboratorio IDOR (controller, router, vistas de lista y perfil)	9 h
Laboratorio SSRF (controller, router, vista, endpoints con flags)	10 h
Laboratorio JWT Attacks (controller, router, vistas de login y admin)	14 h
Diccionario de ciberseguridad (dataset, componente React, vista EJS)	8 h
Estilos CSS globales (temática, grid, tarjetas, alertas, responsive)	12 h
Configuración de seguridad de la plataforma (Helmet, CORS, rate limiting)	4 h
Pruebas funcionales de todos los laboratorios y depuración	12 h
Redacción de la memoria del TFG	24 h
TOTAL REAL	146 h

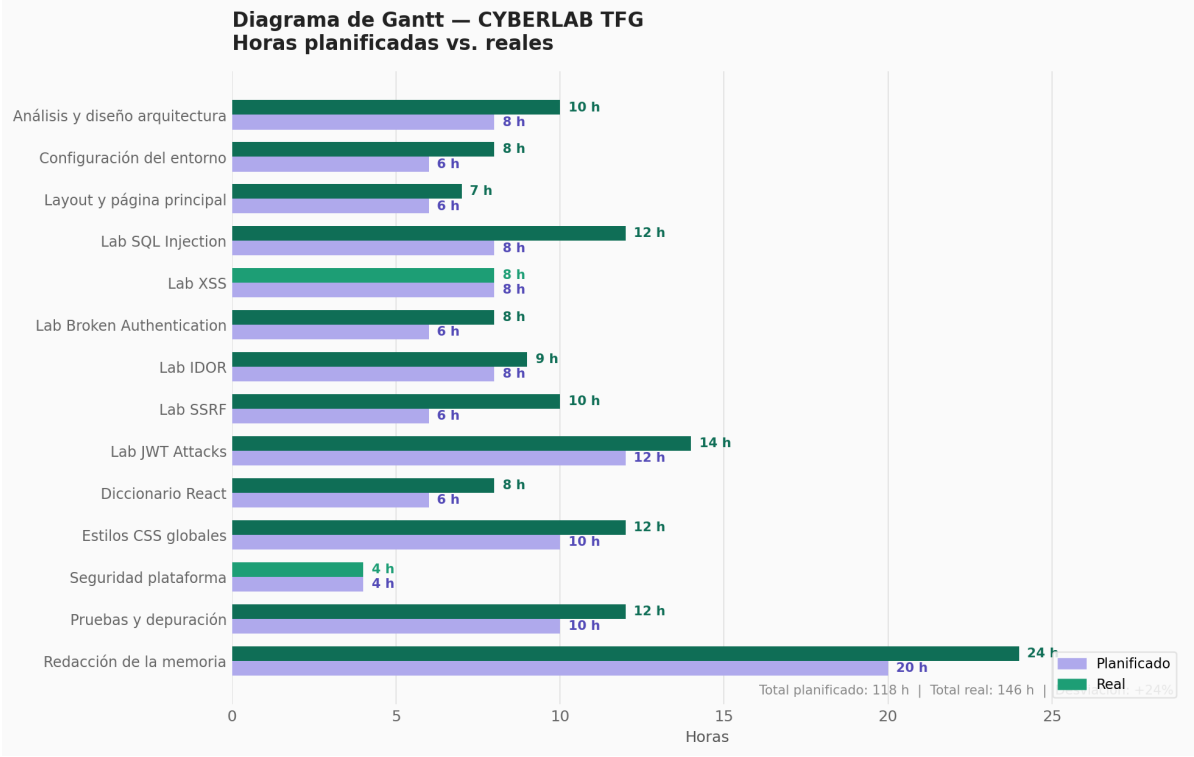
11.4 Análisis de desviaciones

La desviación total ha sido de +20 horas sobre la estimación inicial, lo que representa un incremento del 22% sobre el tiempo previsto. A continuación, se detallan las causas principales de cada desviación:

Tarea con desviación	Desviación	Causa principal
Análisis y diseño de la arquitectura	2 h	La estructura de carpetas y la configuración de express-ejs-layouts requirió más investigación de la esperada para compatibilizar los layouts con los laboratorios que no usan layout.
Configuración del entorno	2 h	Problemas de compatibilidad entre versiones de Sequelize y el driver de base de datos seleccionado.
Layout y página principal	1 h	El diseño responsive de la cuadrícula de laboratorios requirió más iteraciones visuales de las previstas.
Laboratorio SQLi	4 h	La configuración del modelo Sequelize y la sincronización automática de la tabla generó errores en el primer arranque que requirieron depuración.
Laboratorio Broken Auth	2 h	La lógica de bloqueo de cuenta y su persistencia en el array en memoria durante la sesión del servidor presentó comportamientos inesperados que requirieron revisión.
Laboratorio IDOR	1 h	La lógica de detección de manipulación de ID (comparación de lastVisitedId con requestedId) requirió varias iteraciones para funcionar correctamente en todos los casos.
Laboratorio SSRF	4 h	El manejo de respuestas HTML vs. JSON de axios y la presentación del resultado al usuario requirió más trabajo del previsto para ser didáctico.

Laboratorio JWT	2 h	La compatibilidad del parsing de tokens con algoritmo ‘none’ en la librería jsonwebtoken requirió investigación adicional para implementar correctamente el modo vulnerable.
Diccionario (React)	2 h	La sincronización del estado de React con el input nativo del DOM (sin bundler, con React desde CDN) presentó dificultades específicas.
CSS global	2 h	Ajustes de consistencia visual entre el layout compartido y las vistas individuales de laboratorio que no usan el layout.
Pruebas y depuración	2 h	La detección de los banners de explotación (xssTriggered, idorExploited, jwtExploited) requirió depuración extensa para cubrir todos los casos de uso.
Redacción de la memoria	4 h	La creación de los diagramas de los labs y de los flujos de trabajo requirió mas tiempo del esperado.

11.5 Diagrama de Gantt



11.6 Presupuesto estimado

Concepto	Precio/ Hora	Total
Analisis y diseño		
Analisis de requisitos y diseño de arquitectura MVC	25 €/h	250,00 €
Diseño de base de datos y diagrama E/R	25 €/h	75,00 €
Subtotal análisis y diseño	-	325,00 €
Configuración e infraestructura		
Configuración entorno Node.js, Express, EJS, Sequelize, dotenv	20 €/h	160,00 €
Seguridad de la plataforma (Helmet, CORS, rate limiting, sesiones)	25 €/h	100,00 €
Configuración base de datos Neon (PostgreSQL en la nube)	20 €/h	60,00 €
Subtotal infraestructura	-	320,00 €
Desarrollo frontend		
Layout compartido, pagina principal y navegacion	20 €/h	140,00 €
Estilos CSS globales (dark theme, grid, responsive)	20 €/h	240,00 €
Diccionario React (componente, búsqueda, filtros por categoría)	25 €/h	200,00 €
Subtotal frontend	-	580,00 €
Desarrollo de laboratorios		
Laboratorio SQL Injection (router, controller, vista, BD)	30 €/h	360,00 €
Laboratorio XSS (router, controller,vista)	30 €/h	240,00 €

Laboratorio Broken Authentication (bcrypt, bloqueo de cuenta)	30 €/h	240,00 €
Laboratorio IDOR (control de acceso, perfiles)	30 €/h	270,00 €
Laboratorio SSRF (axios, blacklist, endpoints con flags)	30 €/h	300,00 €
Laboratorio JWT Attacks (HS256, algoritmo none, panel admin)	35 €/h	490,00 €
Subtotal Laboratorios	-	1900,00 €
Pruebas y documentación		
Pruebas funcionales, depuración y casos de uso	20 €/h	240,00 €
Redacción de documentación técnica y memoria	15 €/h	360,00 €
Subtotal Pruebas y documentación	-	600,00 €
Licencias y herramientas		
Herramientas de desarrollo (VS Code, Postman) - licencia gratuita	-	0,00 €
Dependencias npm (todas open source)	-	0,00 €
Base de datos Neon (plan gratuito)	-	0,00 €
Repositorio GitHub (plan gratuito)	-	0,00 €
Subtotal licencias	-	0,00 €
TOTAL ESTIMADO (150 h x tarifa media ~24 €/h)	~24 €/h	3725,00 €

Los laboratorios llevan tarifa más alta (30–35 €/h) porque implican conocimiento específico de ciberseguridad, no solo desarrollo web estándar. El JWT se penaliza un poco más por la complejidad criptográfica. La documentación y memoria van a tarifa más baja (15 €/h) porque es trabajo de redacción, no de programación. Las herramientas y licencias salen a 0 € porque el stack completo (Node, npm, Neon free tier, GitHub free, VS Code) es open source o gratuito, lo cual es un punto fuerte del proyecto que conviene destacar.

11.7 Despliegue e instalación

El proyecto incluye un fichero README.md en la raíz del repositorio con un pequeño resumen de lo que es la aplicación y a quien va dirigido y las instrucciones de instalación y arranque de la plataforma. El repositorio GitHub está disponible en https://github.com/Javip1991/Proyecto_TFG_LAB y contiene el historial completo de commits del desarrollo, incluyendo mensajes descriptivos que permiten trazar la evolución del proyecto.

Las **instrucciones** de instalación básicas incluidas en el README son las siguientes:

1. Verificar que Node esta instalado abriendo un terminal y ejecutando `node --version` , donde debería mostrar Versión 18.0.0 o superior.
2. Clonar el repositorio desde la terminal de VScode con el comando `git clone https://github.com/Javip1991/Proyecto_TFG_LAB`
3. Acceder a la carpeta del repositorio a través del comando `cd Proyecto_TFG_LAB.`
4. Instalar las dependencias con `npm install.`
5. Crear el fichero de variables de entorno copiando la plantilla `cp .env.example .env`
6. Editar el archivo `.env` con los valores correctos.
7. Arrancar el servidor con `npm run dev.`

8. Abrir el navegador y acceder a la dirección https://localhost:3000. Debería aparecer la pagina principal de CYBERLAB.

Requisitos mínimos del sistema y dependencias:

Requisitos de hardware mínimos:

- Procesador: cualquier CPU moderna de 64 bits.
- RAM: 512 mínimo, 1GB recomendado.
- Espacio en el disco: 200MB para la aplicación y dependencias

Requisitos de software:

- Sistema operativo: Windows 10/11, macOS 12 o superior, o cualquier distribución de Linux moderna.
- Node.js versión 8 o superior.
- Npm versión 8 o superior (se instala junto a Node.js).
- Git (para clonar el repositorio).
- Navegador web moderno: Chrome 90+, Firefox 88+, Edge 90+ o Safari14+.
- Conexión a internet active durante la ejecución, ya que la base de datos esta alojada en Neon (PostgreSQL en la nube) y React se carga desde CDN.

Dependencias principales del proyecto (Instaladas automáticamente con npm install):

- Express
- Ejs
- Sequelize
- pg (driver de postgreSQL para Node.js, requerido por Sequelize para conexión con Neon)
- pg-hstore (Serialización de objetos JSON para PostgreSQL)
- bcrypt
- jsonwebtoken
- axios

- helmet
- express-session
- express-rate-limit
- dotenv

Configuración necesaria:

La configuración necesaria explica que valores hay que configurar antes de lanzar la aplicación y que significa cada uno. La aplicación necesita un archivo `.env` en la raíz del proyecto con las siguientes variables:

- `PORT`: es el puerto en el que escuchará el servidor. Por defecto será el 3000 pero si esta ocupado se puede cambiar por otro libre.
- `DB_NAME`: nombre de la base de datos creada en Neon.
- `DB_USER`: nombre del usuario de la base de datos de Neon.
- `DB_PASS`: cadena secreta para firmar las cookies de sesión. Debe ser larga y aleatoria.
- `DB_HOST`: dirección del servidor de Neon donde está alojada la base de datos.
- `DB_PORT`: puerto en el que se encuentra alojada la base de datos de la aplicación.
- `JWT_SECRET`: clave secreta para firmar los tokens JWT del laboratorio. En el laboratorio se usa una clave débil intencionadamente para demostrar la vulnerabilidad del algoritmo none.
- `DATABASE_URL`: cadena de conexión completa proporcionada por Neon. Se obtiene desde el panel de Neon en “Conecction Details”. Incluye usuario, contrraseña, host, nombre de la base de datos y el parámetro `sslmode=none` obligatorio para conectar con Neon de forma segura.

Configuración de Sequelize con Neon:

La conexión con Neon requiere que SSL este habilitado explícitamente en la configuración de Sequelize. En el fichero config/database.js debe estar configurado de la siguiente forma:

```
const { Sequelize } = require("sequelize");
require("dotenv").config();

const sequelize = new Sequelize(process.env.DATABASE_URL, {

    dialect: "postgres",
    logging: false,
    dialectOptions: {
      ssl: {
        require: true,
        rejectUnauthorized: false,
      },
    },
    ssl: true,
  });

module.exports = sequelize;
```

Planes de contingencia (que hacer si algo falla):

Problema 1: El servidor no arranca – error de puerto ocupado

- Síntoma: aparece un mensaje de error referido al puerto.
- Solución: Cambiar el valor de PORT en el fichero .env a otro puerto disponible, por ejemplo el 3001, y volver a lanzar la aplicación.

Problema 2: Error de conexión con Neon

- Síntoma: aparece el mensaje SequelizeConnectionError o connection refused al arrancar el servidor.
- Solución: verificar los siguientes puntos en orden:
    - Comprobar que la variable DATABASE_URL en el fichero .env es correcta y esta completa, incluyendo ?sslmode_require al final.
    - Verificar que hay conexión a internet activa, ya que Neon es una base de datos en la nube.
    - Acceder al panel de Neon en https://console.neon.tech y comprobar que el proyecto esta activo y no suspendido. Neon suspende

automáticamente los proyectos inactivos en el plan gratuito. Basta con acceder al panel para reactivarlo.

- Verificar que SSL esta correctamente configurado en config/database.js con rejectUnauthorized: false.

Problema 3: La tabla lab_sqli_users no existe o esta vacia:

- Síntoma: el laboratorio de SQL Injection da error o no muestra los resultados al hacer Login.
- Solución: Sequelize sincroniza y crea la tabla automáticamente al arrancar el servidor mediante sequelize.sync(). Si la tabla no se ha creado, revisar que la conexión con Neon es correcta y que en server.js se llama a sequelize.sync() antes de que el servidor empiece a escuchar peticiones. Si la tabla existe pero esta vacía, verificar que el *seed* de datos de prueba se ejecuta correctamente al arrancar.

Problema 4: Error al instalar dependencias con npm install:

- Síntoma: errores durante la instalación, especialmente bcrypt que requiere compilación nativa, o en pg que necesita librerías del sistema.
- Solución: verificar que la versión de Node.js sea 18 o superior con node --version. En Windows puede ser necesario instalar las herramientas de compilación, y en Linux asegurarse de tener instalado build-essential con apt install build-essential.

Problema 5: El diccionario React no carga:

- Síntoma: La pagina del diccionario no aparece o aparece en blanco.
- Solución: Verificar la conexión a internet, ya que React carga desde CDN de Cloudflare y sin conexión activa aparecerá vacío o en blanco.

Problema 6: El modo vulnerable/seguro no se mantiene entre paginas:

- Síntoma: al cambiar de modo en un laboratorio y navegar a otra pagina, el modo vuelve al estado por defecto.
- Solución: verificar que `SESSION_SECRET` esta correctamente definido y que el middleware de sesiones esta cargado antes que los routers en `app.js`.

Problema 7: Neon suspende la base de datos durante una demostración:

- Síntoma: La aplicación deja de responder o muestra errores de conexión de forma inesperada durante el uso normal.
- Solución: esto ocurre en el plan gratuito de Neon cuando la base de datos lleva varios minutos sin actividad. La reactivación es automática en cuanto llega una nueva petición, aunque puede tardar entre 1 y 3 segundos. Si esto ocurre, también se puede recargar la pagina para arreglar el problema.

12. TRABAJOS FUTUROS

CYBERLAB es una plataforma concebida para crecer de forma modular. A continuación, se identifican las principales líneas de trabajo y mejora proyectadas para versiones futuras del proyecto, ordenadas por prioridad e impacto educativo estimado.

12.1. Nuevos laboratorios planificados

- Laboratorio CSRF (Cross-Site Request Forgery): formulario de transferencia bancaria simulada. En modo vulnerable, el formulario no incluye token CSRF y cualquier página externa puede enviar peticiones en nombre del usuario autenticado. En modo seguro, se implementa el patrón de token CSRF sincronizado.
- Laboratorio Path Traversal: exploración de ficheros del sistema mediante secuencias ../ en rutas de archivo. El laboratorio demostrará cómo acceder a ficheros fuera del directorio permitido y cómo prevenirlo mediante path.resolve() y validación de rutas.
- Laboratorio XXE (XML External Entity): procesamiento de XML con entidades externas para leer ficheros del servidor o realizar SSRF. Especialmente relevante para APIs que aceptan XML como formato de entrada.
- Laboratorio Command Injection: ejecución de comandos del sistema operativo a través de campos de entrada en una funcionalidad de 'ping' simulada. Demostración del uso de child_process.exec() con input no sanitizado versus el uso de execFile() con argumentos separados.
- Laboratorio Open Redirect: redirección a URLs controladas por el usuario sin validación, utilizable para ataques de phishing con dominios legítimos como intermediarios.

12.2. Sistema de progreso y gamificación

- Sistema de autenticación real con registro de usuarios para persistir el progreso entre sesiones.
- Registro de laboratorios completados y flags capturados por cada usuario en base de datos.
- Sistema de puntos con ponderación según la dificultad del laboratorio y ranking entre usuarios (tabla de clasificación pública).
- Niveles de dificultad en cada laboratorio: nivel básico (guiado con payloads de ejemplo), nivel intermedio (sin pistas directas) y nivel avanzado (sin descripción del vector de ataque).
- Insignias (badges) desbloqueables al completar grupos de laboratorios relacionados.

12.3. Mejoras técnicas y de despliegue

- Despliegue en producción en servicio cloud (Railway, Render o VPS propio) con dominio personalizado para hacer la plataforma accesible sin instalación local.
- Implementación de HTTPS con certificado TLS mediante Let's Encrypt para garantizar la seguridad de las comunicaciones incluso en el propio entorno educativo.
- Internacionalización (i18n) para añadir soporte en inglés, ampliando el alcance potencial de la plataforma más allá del ámbito hispanohablante.
- Mejora del diseño responsive para dispositivos móviles, donde actualmente algunos laboratorios no presentan una experiencia de usuario óptima.
- Panel de administración para gestión de laboratorios: activar/desactivar laboratorios específicos, configurar el nivel de dificultad por defecto y gestionar usuarios registrados.

- Ampliación del diccionario de ciberseguridad con más de 100 entradas, referencias bibliográficas enlazadas (CVE, OWASP, RFC) y ejemplos de código para cada concepto.

13. CONCLUSIONES

El desarrollo de CYBERLAB ha supuesto un proceso de aprendizaje intenso y enriquecedor en dos ámbitos complementarios: el desarrollo web con Node.js/Express y la ciberseguridad aplicada a aplicaciones web. La plataforma resultante cumple con todos los objetivos principales planteados al inicio del proyecto: implementa laboratorios interactivos funcionales para las seis vulnerabilidades OWASP seleccionadas, con un sistema de modo dual que permite comparar en tiempo real el comportamiento vulnerable y seguro, visualización del código fuente y un diccionario de términos integrado.

Desde el punto de vista técnico, el proyecto ha consolidado competencias avanzadas en el ecosistema Node.js: gestión de middlewares con Express, renderizado dinámico con EJS y layouts compartidos, ORM con Sequelize, hashing de contraseñas con bcrypt, generación y verificación de tokens JWT, peticiones HTTP desde el servidor con axios y técnicas de seguridad a nivel de cabeceras HTTP y gestión de sesiones. La implementación del componente React del diccionario sin bundler ha sido especialmente formativa, al requerir un entendimiento profundo del modelo de componentes de React independiente de las herramientas de build típicas.

La implementación del laboratorio JWT ha resultado particularmente relevante desde el punto de vista profesional, al exigir una comprensión profunda del estándar RFC 7519, los algoritmos de firma criptográfica y las implicaciones de seguridad de cada decisión de implementación. De manera similar, el laboratorio SSRF ha ilustrado cómo una funcionalidad aparentemente inocua —permitir al servidor hacer peticiones a URLs externas— puede convertirse en un vector de ataque de alto impacto en entornos cloud donde los metadatos de instancia son accesibles a través de rutas internas.

El proyecto tiene valor educativo real y puede ser utilizado como recurso complementario en módulos de seguridad informática de ciclos formativos de DAW y ASIR. La arquitectura modular adoptada facilita la incorporación de nuevos laboratorios sin necesidad de re-factorizar el código existente, lo que garantiza la escalabilidad del proyecto a largo plazo y abre la puerta a contribuciones externas mediante el repositorio público de GitHub.

Como reflexión profesional, CYBERLAB ha reforzado la convicción de que la seguridad debe ser una consideración presente desde el inicio del diseño de cualquier aplicación —el principio de security by design— y no un elemento añadido a posteriori una vez que el sistema ya está en producción. Entender el funcionamiento real de los ataques, haberlos reproducido en un entorno controlado y haber implementado sus contramedidas, proporciona una perspectiva que ningún manual teórico puede ofrecer por sí solo. Esta experiencia marcará la forma en que se abordará el desarrollo de software en el ámbito profesional a partir de este proyecto.

14. REFERENCIAS

Todas las referencias se presentan siguiendo las normas APA 7ª edición. Cada referencia va acompañada del apartado del proyecto en el que fue aplicada.

OWASP Foundation. (2021). OWASP Top Ten 2021. Recuperado de https://owasp.org/www-project-top-ten/

Aplicado en la selección y clasificación de las vulnerabilidades implementadas en los laboratorios de CYBERLAB (SQLi, XSS, Broken Auth, IDOR, SSRF, JWT).

PortSwigger Web Security. (2024). Web Security Academy. Recuperado de https://portswigger.net/web-security

Consulta de técnicas detalladas de explotación y vectores de ataque para todos los laboratorios. Referencia principal para los labs de JWT Attacks y SSRF.

OpenJS Foundation. (2024). Express.js Documentation. Recuperado de https://expressjs.com/en/4x/api.html

Referencia principal para la configuración de middlewares, sistema de enrutamiento, motor de vistas y gestión de peticiones y respuestas HTTP.

Sequelize. (2024). Sequelize v6 Documentation. Recuperado de https://sequelize.org/docs/v6/

Consulta de la API de Sequelize para definición de modelos, consultas parametrizadas con replacements, sincronización del esquema y autenticación de la conexión.

npm. (2024). jsonwebtoken (v9). Recuperado de https://www.npmjs.com/package/jsonwebtoken

Documentación de la librería utilizada en el laboratorio JWT para la firma de tokens HS256 con jwt.sign() y la verificación estricta con jwt.verify() y la opción algorithms.

npm. (2024). bcrypt (v5). Recuperado de https://www.npmjs.com/package/bcrypt

Documentación de la librería de hashing de contraseñas utilizada en el laboratorio de Broken Authentication para la generación de hashes con bcrypt.hashSync() y la comparación con bcrypt.compare().

npm. (2024). helmet (v7). Recuperado de https://www.npmjs.com/package/helmet

Referencia para la configuración del middleware de cabeceras de seguridad HTTP aplicado globalmente en app.js.

npm. (2024). axios (v1). Recuperado de https://www.npmjs.com/package/axios

Documentación del cliente HTTP utilizado en el laboratorio SSRF para realizar peticiones desde el backend del servidor a las URLs proporcionadas por el usuario.

Jones, M. B., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) — RFC 7519. IETF. Recuperado de https://datatracker.ietf.org/doc/html/rfc7519

Estándar oficial del formato JWT consultado para la implementación correcta del laboratorio de JWT Attacks: estructura del token, algoritmos de firma soportados y el problema del algoritmo 'none'.

Meta Open Source. (2024). React Documentation. Recuperado de https://react.dev/reference/react

Referencia para la implementación del componente React del diccionario sin bundler, utilizando los hooks useState y useEffect para gestionar el estado de búsqueda y filtrado.

MITRE Corporation. (2024). Common Vulnerabilities and Exposures (CVE). Recuperado de https://cve.mitre.org/

Consulta del sistema de identificación de vulnerabilidades para las definiciones incluidas en el diccionario de ciberseguridad de la plataforma.

OWASP Foundation. (2023). Testing Guide v4.2. Recuperado de https://owasp.org/www-project-web-security-testing-guide/

Referencia metodológica para el diseño de los casos de prueba de explotación de cada laboratorio.

npm. (2024). express-session (v1). Recuperado de https://www.npmjs.com/package/express-session

Documentación del middleware de sesiones HTTP utilizado para persistir el modo activo (vulnerable/seguro) de cada laboratorio entre peticiones HTTP.